20.000 Güncel arXiv Özeti Üzerinde Yüksek Başarımlı Benzerlik Araması

YZM 513 – Yapay Zekâ için Yüksek Başarımlı Programlama · Proje 1 *Ahmet Ozan Yıldırım (269797005) · Mazhar Selvitopi (269797013) · GitHub: ahmetozanyildirim/yzm513-similarity-search*

> **Özet.** Görev: 768 boyutta gömülmüş 20.000 arXiv özeti üzerinde, 200 sorgunun her biri için kosinüs benzerliğine göre en benzer 10 özeti döndürmek. Saf Python baseline **154,4 s** sürüyor. Son motor **3,89 ms (39.689x daha hızlı)**, tek bir sorgu için ise 0,21 ms sürüyor. Hattı: vektörleri bir kez normalize et → TruncatedSVD ile 128 boyuta indir → tek bir batch'li BLAS matris çarpımı → 100 adayın paralel top-k seçimi → 768 boyutta kesin re-rank. Her corpus boyutunda her sorgu için baseline'ın top-10'unu, skorlar 7,7e-8 içinde olacak şekilde döndürüyor. Üç ders:
>
> - **Python içindeki düzeltmeler en fazla 3,8x'e ulaşıyor.** Baseline'ı yavaşlatan algoritma değil, yorumlayıcı.
> - **Vektörizasyondan sonra darboğaz tek thread'li top-k seçimine kaydı.** Sürenin %66'sını alıyordu ve BLAS thread'i eklemek bunu çözemedi: Amdahl yasası gereği speed-up 16 thread'de 2,1x'te duruyor.
> - **Tek process içindeki thread'ler process havuzlarını, Dask'ın process ve dağıtık zamanlayıcılarını ve Ray'i geçiyor.** Bu boyutta veri RAM'e, hatta 96 MB'lık L3 cache'e sığıyor.

1. Giriş

1.1 Problem

Kullanıcı bir sorgu gönderir; sistem, **N = 20.000** güncel makale özetinden oluşan bir corpus içinden **en benzer k (k = 10) dokümanı** döndürmelidir. Etkileşimli kullanıma yetecek kadar hızlı olmalıdır: sorgu başına 100 ms'nin çok altında. Benzerlik, 768 boyutlu cümle embedding'lerinin kosinüsüdür. Doğrudan yazılmış saf Python bir uygulama **sorgu başına 0,77 s** sürüyor (Bölüm 2). Bu rapor o baseline'dan milisaniyenin altındaki bir motora giden yolu izliyor; yoldaki her tasarım kararı bir ölçümle destekleniyor. Her optimize sürüm, skorlar 1e-5 içinde olmak üzere **baseline ile aynı top-10'u** döndürmek zorunda.

1.2 Veri seti: arXiv özetleri 2025-2026 (Seçenek B)

B seçeneğini (güncel akademik özetler) seçtik, çünkü **gerçekçi bir arama iş yükü** (makale arama, bir RAG sisteminin geri getirme adımı), **ücretsiz ve tekrarlanabilir** (herkese açık arXiv API, `scripts/prepare_data.py`) ve **içinde hazır bir doğruluk sinyali** var: sorgular makale başlıkları, her makalenin kendi özeti de bilinen doğru cevap. A ve C seçenekleri ücretli API çağrıları ya da çalışan bir sistem gerektirirdi; D seçeneği ise sentetik verinin gerçek bir iş yükünü temsil ettiğine dair bir gerekçe isterdi.

Kaynak. arXiv API; cs.AI, cs.CL ve cs.LG kategorileri; Ocak 2025 ile Eylül 2026 arasında gönderilen makaleler: 21 ay x 3 kategori = her biri 500 makalelik 63 (ay, kategori) hücresi, toplam 31.500 kayıt.

Ön işleme adımı	Kalan kayıt	Neden
Ham API kayıtları	31.500	
Tekrarlanan arXiv ID'lerini çıkar	24.866	Birden fazla kategoride listelenen bir makale birkaç hücrede görünüyor (kayıtların ~%21'i).
Birebir aynı özetleri çıkar	24.865	
>= 50 kelimelik özetleri tut	24.833	Ödev ortalama >= 50 token istiyor; bu adım 32 çok kısa kaydı atıyor.
Rastgele örneklem (seed 42)	**20.000**	Her önek (ilk 1K, 5K, 10K) de rastgele bir örneklem; ölçekleme testinde kullanılıyor.

Kayıtlar ayrıştırılırken boşluklar tek boşluğa indiriliyor. **Doküman yalnızca özettir**: başlık dışarıda tutuldu, yoksa başlıkla arama önemsiz derecede kolay olurdu. **200 sorgu**, ilk 1.000 dokümandan seçilen 200 makalenin başlıklarıdır; böylece doğru cevap ölçekleme testinin her corpus boyutunda mevcuttur.

İstatistik	Değer
Doküman / sorgu	20.000 / 200
Özet uzunluğu (modelin tokenizer'ı)	Ortalama **261** token, medyan 258, en az 59, en çok 1.052 (ortalama 178 kelime)
Modelin 384 token'lık penceresinden uzun özetler	%3,8 (embedding modeli sonlarını kesiyor)
Sorgu uzunluğu / tarihler	Ortalama 10,4 kelime / 2025-01-01 ile 2026-09-05 arası
Ana kategoriler (en sık)	cs.CL 5.869 · cs.LG 5.212 · cs.AI 2.303 · cs.CV 1.786 · cs.CR 515 · cs.RO 416 · stat.ML 381 · cs.IR 338

Örnek kayıtlar (her sorguyu hedef dokümanı izliyor; daha fazla istatistik ve Şekil 0 Ek A'da):

Sorgu (başlık)	Hedef özet (başı)
Distributed Equivariant Graph Neural Networks for Large-Scale Electronic Structure Prediction	Doküman 6, cs.LG, 2025-07-04, 140 kelime: "Equivariant Graph Neural Networks (eGNNs) trained on density-functional theory (DFT) data can potentially perform electronic structure prediction at unprecedented scales, ..."
COFFEE: COdesign Framework for Feature Enriched Embeddings in Ads-Ranking Systems	Doküman 22, cs.IR, 2026-01-06, 206 kelime: "Diverse and enriched data sources are essential for commercial ads-recommendation models to accurately assess user interest ..."

Vektörler. Her özet ve sorgu `sentence-transformers/all-mpnet-base-v2` ile gömülüyor (768 boyut, float32, birim uzunluk): corpus matrisi 20.000 x 768 = **58,6 MB**. 768, izin verilen aralığın üst ucu; bu da boyut indirgemenin etkisini net biçimde görünür kılıyor. Ödevin FAQ'sında belirtildiği gibi embedding üretimi hiçbir benchmark'ın parçası **değil**; bir kez yapılıp cache'lendi (bu CPU'da 2.199 s).

Sağlama. 20.000 özetin tamamında kesin kosinüs aramasıyla, başlığın kendi özeti sorguların **%74**'ünde birinci sırada, **%90**'ında ilk 10 içinde: embedding'ler anlamlı ve görev önemsiz değil (kaçırılanlar çoğunlukla çok genel başlıklar ya da aynı konuda yakın komşuları olan makaleler).

1.3 Donanım ve yazılım

Bileşen	Sürüm / özellik
CPU	AMD Ryzen 7 9800X3D: 8 fiziksel çekirdek / 16 thread (SMT), **96 MB L3 cache** (3D V-Cache)
RAM / İşletim sistemi	31,2 GB / Windows 11 (10.0.26200)
Python	3.12.10 (sanal ortam, `requirements.txt`); bir deney için free-threaded 3.14.3t
Sayısal yığın	NumPy 2.5.4 (OpenBLAS 0.3.34 ile, 16 thread), SciPy 1.18.1, scikit-learn 1.9.1
JIT / paralellik	Numba 0.68.0 (16 thread), Dask 2026.8.0, Ray 2.59.0, FAISS 1.15.1
Profiling	cProfile, line_profiler 5.1.0, py-spy

L3 cache'in boyutu ileride birkaç sonucu açıklıyor: 58,6 MB'lık corpus ona sığıyor, 10 kat büyüğü ise sığmıyor (Bölüm 6).

1.4 Ölçüm yöntemi

- Her ölçümde önce **warm-up çalıştırmaları** (en az bir; JIT ile derlenen her şeyde üç), ardından `time.perf_counter` ile **en az 5 ölçümlü çalıştırma** yapılıyor; **medyan** raporlanıyor (ortalama, std ve ham süreler `results/` altında).
- **Ekstrapolasyon yok:** her süre, belirtilen iş yükünün doğrudan ölçümü. Naif baseline tam olarak ölçüldü: 1K, 5K, 10K ve 20K dokümanda 200 sorgunun tamamı, her biri 1 warm-up + 5 ölçüm.
- **Tek temiz çalıştırma:** bütün fazlar boş bir makinede, sırayla, bir kez çalıştı (toplam 70 dk); çalıştırma kaydı hiçbir fazdan önce başka meşgul bir process göstermedi.
- **Doğruluk:** her optimize sürüm, saklanan baseline top-10'u ile sıra sıra karşılaştırılıyor; skorlar **1e-5** içinde.
- Bir karşılaştırma gerektirdiğinde BLAS ve Numba thread sayıları açıkça ayarlanıyor. Tek seferlik maliyetler (embedding, normalizasyon, SVD fit, ters indeks) ayrı raporlanıyor, arama sürelerine asla dahil edilmiyor. Her şekil ve sayı `src/hpss/experiments/` tarafından üretiliyor (`python scripts/run_all.py`).

2. Baseline ve Profiling

2.1 Naif algoritma

Baseline ödevi harfiyen izliyor: saf Python, iç içe döngüler ve eleman eleman hesaplanan kosinüs (`src/hpss/naive.py`).

```
def cosine_similarity_python(a, b):
    dot = 0.0
    norm_a = 0.0
    norm_b = 0.0
    for i in range(len(a)):
        dot += a[i] * b[i]
        norm_a += a[i] * a[i]
        norm_b += b[i] * b[i]
    if norm_a == 0.0 or norm_b == 0.0:
        return 0.0
    return dot / (math.sqrt(norm_a) * math.sqrt(norm_b))

def search_naive(queries, corpus, k=10):
    results = []
    for query in queries:
        scores = []
        for d_idx, doc in enumerate(corpus):
            sim = cosine_similarity_python(query, doc)
            scores.append((d_idx, sim))
        scores.sort(key=lambda x: x[1], reverse=True)
        results.append(scores[:k])
    return results
```

Kod: `src/hpss/naive.py` – `cosine_similarity_python` + `search_naive` (L18–46); §2.5'teki saf Python düzeltmeleri: `search_python_prenorm`, `search_python_builtin` (L53–85).

200 sorgu ve 20.000 doküman için bu, kosinüs fonksiyonunun 4 milyon kez çağrılması demek. Her çağrı üç çarpma-toplamalı 768 döngü turu çalıştırıyor; bu da **yorumlayıcıdan geçen 9,2 milyar çarpma-toplama** ediyor. Bunların üçte ikisi yalnızca normları yeniden hesaplıyor.

2.2 Baseline süresi

Her corpus boyutunda 200 sorgunun tamamı, her biri 1 warm-up + 5 ölçüm (hiçbir şey ekstrapole edilmedi):

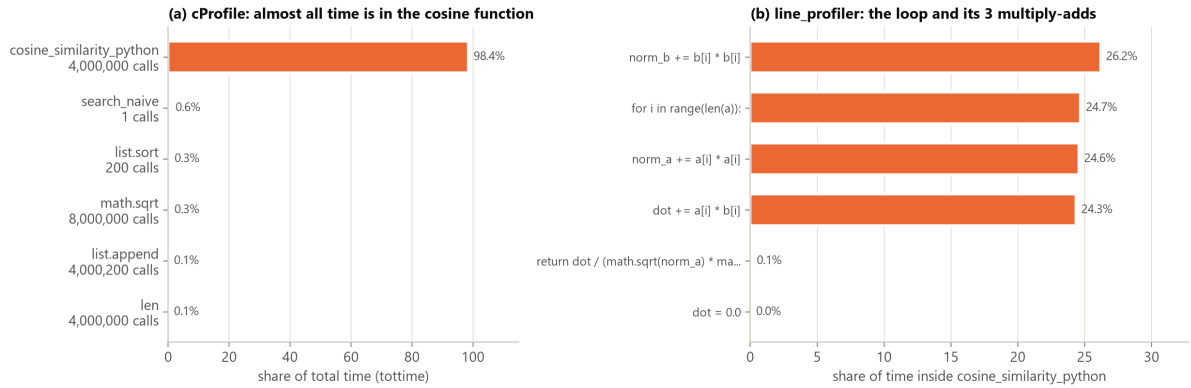
Doküman	Medyan (5 ölçüm)	Std	Sorgu başına	Sorgu-doküman çifti başına
1.000	7,73 s	0,02 s	38,6 ms	38,6 µs
5.000	38,62 s	0,10 s	193 ms	38,6 µs
10.000	77,59 s	0,39 s	388 ms	38,8 µs
20.000	**154,36 s**	0,07 s	**772 ms**	38,6 µs

Baseline: 200 sorgu x 20.000 doküman 154,4 s sürüyor, yani sorgu başına 0,77 s; bu, 100 ms'lik etkileşim bütçesinin neredeyse sekiz katı. Süre tam olarak doğrusal büyüyor (20x doküman, 19,98x süre; çift başına maliyet %0,5 içinde sabit): sorun karmaşıklık, O(Q·N·d), değil; onun sabit çarpanı. Tek bir kosinüs çağrısının `timeit` ölçümü **38,3 ± 0,4 µs**, çift başına maliyetle aynı; yani sürenin neredeyse tamamı bu tek fonksiyonun içinde geçiyor.

2.3 Profiling

Hafta 1'in dört aracı, her biri farklı bir soruyu cevaplıyor:

- **`timeit`** – tek bir kosinüs çağrısının maliyeti (yukarıda).
- **`cProfile`** – *hangi fonksiyon*: baseline'ın tam iş yükü, ayrı ve düz bir Python process'inde bir kez profillendi (`scripts/profile_naive.py --cprofile`). Python 3.12'den beri cProfile, process'in her thread'ini gören `sys.monitoring` üzerine kurulu; Jupyter içinde kernel'in kendi I/O thread'leri profile karıştı (Bölüm 7).
- **`line_profiler`** – *hangi satır*: bu kodu 12,8 kat yavaşlattığı için 20.000 dokümanlık tam corpus'a karşı 10 sorguyla çalışıyor (200.000 çağrı). Sürenin paylarını raporluyor; paylar sorgu sayısına bağlı değil.
- **`py-spy`** – kod değişikliği gerektirmeyen, örnekleme yapan bir profiler: tam iş yükünün alev grafiği (`results/1_baseline/flamegraph.svg`).



Şekil 1: Naif arama sürenin %98,4'ünü `cosine_similarity_python` içinde (her sorgu-doküman çifti için bir tane, 4.000.000 çağrı), yalnızca %0,4'ünü sıralamada geçiriyor. Fonksiyonun içinde döngü başlığı ve üç çarpma-toplama satırı neredeyse eşit paylarla sürenin %99,8'ini alıyor. Maliyet, her float işlemindeki yorumlayıcı yükü; üç çarpma-toplamadan ikisi de yalnızca normları yeniden hesaplıyor.

- **cProfile:** `cosine_similarity_python`'ın kendi süresi 190,1 s'nin 187,1 s'si (**%98,4**); sıralama (`list.sort` + anahtar `lambda`'sı) %0,4, `math.sqrt` %0,3, `list.append` %0,1. Profiler ek yükü yalnızca 1,23x (190,1 s'ye karşı 154,4 s), çünkü profillenen her çağrı 768 döngü turu gerçek iş yapıyor.
- **line_profiler:** döngü başlığı ve üç çarpma-toplama satırı her biri %24,3–26,2, birlikte **%99,8** alıyor. `search_naive` içinde kosinüs fonksiyonunu çağıran satır %99,85; `append` ve `sort` her biri %0,1'in altında.

2.4 En büyük 3 darboğaz

#	Darboğaz	Kanıt
1	**Her float işleminin yorumlayıcı yükü.** Döngünün her satırı, iki listeyi indeksleyen, yeni bir `float` nesnesi oluşturan ve işlemi yönlendiren bir bytecode adımı.	Döngünün dört satırı, işe yarar iş yapsın yapmasın aynı maliyette: çalıştırma başına ~12,6 ns (38,6 µs / 768 tur / 4 satır). İşe yarar hız **saniyede 20 milyon çarpma-toplama**. Bu CPU'nun tek çekirdeği BLAS matris çarpımını 312 GFLOP/s'de çalıştırıyor (Ek C): 7.800x fazlası.
2	**Gereksiz iş.** İki norm da her çift için yeniden hesaplanıyor: sorgu normu sorgu başına 20.000 kez, her doküman normu 200 kez.	Eşit maliyetli dört döngü satırından ikisi (sürenin ~%50'si) norm hesaplıyor.
3	**Veri düzeni ve tam sıralama.** Her sayı, bir işaretçinin arkasında 24 baytlık ayrı bir Python nesnesi. Ardışık bellek ve SIMD yok, cache'ler kötü kullanılıyor. Yalnızca 10'u gerektiği halde 20.000 skorun tamamı sıralanıyor, O(N log N).	Bir vektör Python listesi olarak 24 KB, float32 olarak 3 KB yer kaplıyor (8x); corpus 470 MB'a karşı 58,6 MB. Sıralama bugün %0,4, ama skorlama hızlanınca payı büyüyor (Bölüm 6.1).

Zaman nereye gidiyor? Sıralamaya (< %1) değil, benzerlik hesabına (%98). Benzerlik hesabının içinde de sayıların *nasıl* işlendiğine: yorumlayıcı tarafından, bellekte dağınık duran kutulanmış nesneler olarak.

2.5 Hipotezlerin saf Python'da testi

Saf Python'da kalarak her darboğazı kaldırdık. Her sürüm aynı tam iş yükünü çalıştırdı (1 warm-up + 5 ölçüm) ve baseline'a karşı kontrol edildi:

Sürüm	200 sorgu x 20K doküman (medyan)	Speed-up	Doğru
Naif baseline	154,36 s	1,0x	✓
Düzeltme 2: normlar bir kez (vektörler indeksleme sırasında normalize)	66,44 s	2,3x	✓
Düzeltme 1+3: `sum(map(mul, q, d))` + `heapq.nlargest`	40,39 s	3,8x	✓
NumPy, 1 thread, float64 (referans)	65,6 ms	2.353x	✓

Bir kez normalize etmek döngü gövdesini dört satırdan ikiye indiriyor ve 2,3x kazandırıyor; darboğaz 2 doğrulanıyor. Döngüyü C'de yazılmış yerleşik fonksiyonlara taşımak ve sıralamak yerine top-k'yı seçmek 3,8x veriyor; saf Python'un gidebileceği yer aşağı yukarı bu. **Aynı tek çekirdekte**, aynı algoritmayla NumPy, en iyi saf Python sürümünden hâlâ **616x daha hızlı**: kalan fark darboğaz 1 ve 3, yani yorumlayıcı ile kutulanmış, dağınık veri. "Normlar bir kez" sürümünü yeniden profillemek bunu doğruluyor (nokta çarpımı satırı %52,4, döngü başlığı %47,1, sıralama %0,2). Yorumlayıcı Python'un içinde düzeltilemez; bu yüzden Bölüm 3 onu geride bırakıyor. Baseline'ın dört boyuttaki top-10 listeleri (`results/1_baseline/baseline_topk.npz`), sonraki her sürümün doğruluk referansı.

3. Matris İşlemleri ve Boyut İndirgeme

3.1 Sparse ve dense temsil

TF-IDF. Aynı metinlerin sparse bir temsili: scikit-learn `TfidfVectorizer`, 10.000 terimlik sözlük (`min_df=2`), L2-normalize satırlar; böylece kosinüs nokta çarpımına eşit. 20.000 x 10.000 matriste 1.832.911 sıfır olmayan eleman var: **yoğunluk %0,92** (özet başına 91,6, sorgu başına 8,0 terim).

Üç Numba kernel'i yazdık; `prange` kullanıyorlar, ama tek thread'li kütüphanelerle adil karşılaştırma için Numba'nın thread sayısı **bire** ayarlı. **sparse·sparse** bir ters indeks kullanıyor: CSR biçiminde X^T, yani her terim için (doküman, ağırlık) çiftlerinin listesi; indeksleme sırasında bir kez kuruluyor (7,4 ms). Yalnızca sorguyla ortak terimi olan dokümanlara dokunuyor, dolayısıyla hiçbir sıfırı çarpmıyor. **sparse·dense**, X'in CSR satırlarını dense sorgularla çarpıyor; **dense·dense** sıfırlar dahil dense matrisler üzerinde dönüyor.

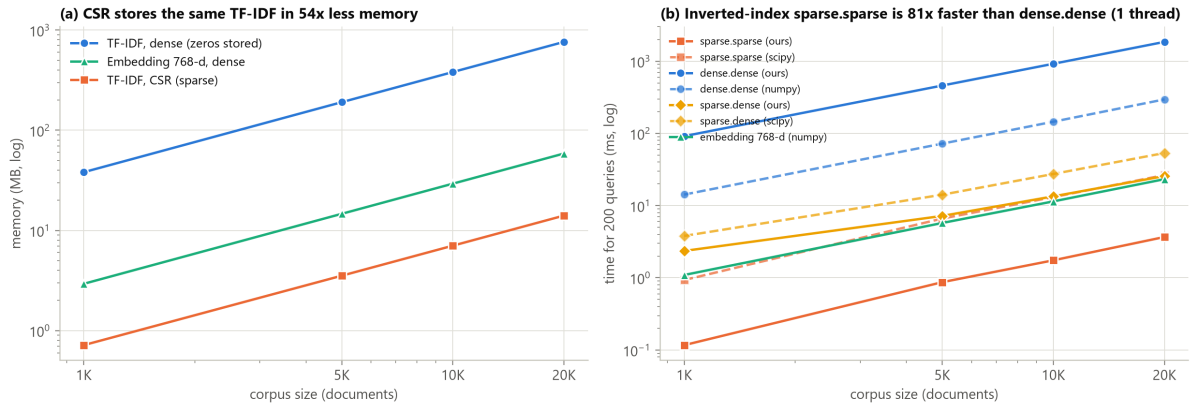
```
@njit(parallel=True, cache=True)
def _spsp_kernel(q_indptr, q_indices, q_data, inv_indptr, inv_docs, inv_vals, n_docs):
    S = np.zeros((q_indptr.shape[0] - 1, n_docs), dtype=np.float32)
    for i in prange(q_indptr.shape[0] - 1):
        for p in range(q_indptr[i], q_indptr[i + 1]):          # each term t of query i ...
            t, w = q_indices[p], q_data[p]
            for r in range(inv_indptr[t], inv_indptr[t + 1]):  # ... and each document that contains
t
                S[i, inv_docs[r]] += w * inv_vals[r]
    return S
```

Üç kombinasyonun kodu: `src/hpss/sparse_dense.py` – sparse·sparse `_spsp_kernel` + `build_inverted_index` (L46–67, yukarıda), sparse·dense `_spdn_kernel` (L74–90), dense·dense `_dndn_kernel` (L97–112); kütüphane referansları `ref_*` (L119–128).

200 sorgu x 20.000 doküman, 1 thread	Bizim	Kütüphane	X'in belleği
sparse·sparse (ters indeks)	**3,67 ms**	26,4 ms (SciPy CSR @ CSR)	14,1 MB (CSR)
sparse·dense	25,6 ms	53,5 ms (SciPy)	14,1 MB
dense·dense (sıfırlarıyla TF-IDF, 10.000-d)	1.851 ms	296 ms (BLAS)	763 MB
dense embedding'ler (768-d)	–	23,2 ms (BLAS)	58,6 MB

CSR yalnızca sıfır olmayanları saklıyor (değer 4 B + sütun indeksi 4 B, artı satır başına bir satır işaretçisi): dense TF-IDF dizisinden **54x daha az bellek**. Ters indeksimiz SciPy'nin genel sparse çarpımından (ki o sparse bir sonuç matrisi de kurmak zorunda) 7x, dense TF-IDF matrisi üzerindeki BLAS'tan 81x ve 768-d embedding'leri skorlamaktan 6x daha hızlı. Sparse yapı yalnızca çok sparse veride kazandırıyor: rastgele matrislerde kernel'imiz tek thread'li BLAS'ı %10 yoğunluğa kadar yeniyor (62,5'e karşı 72,0 ms), ama %20'de açıkça kaybediyor (244'e karşı 73 ms; Şekil 2b, Ek D). %0,92 yoğunluktaki TF-IDF'imiz sparse tarafın çok içinde.

Kalite belirleyici. Hız ve bellek TF-IDF'ten yana, ama embedding doğru özeti daha sık buluyor (hit@1 %74'e karşı %68,5; hit@10 ikisinde de %90) ve dokümanlarla hiç anahtar kelime paylaşmayan yeniden ifadeleri de eşleştiriyor. *"speeding up text generation of large language models"* sorgusunda TF-IDF kelimeleri eşleştirip ilk sırada bir SSD harici bellek arama makalesi döndürüyor; embedding ise *"Mamba Drafters for Speculative Decoding"* döndürüyor – tam olarak sorgunun anlamı. **Karar: dense 768-d embedding'ler.** Aramaları tek thread'de 23 ms sürüyor ve raporun geri kalanı onu hızlandırıyor; bir üretim sistemi ikisini birleştirebilir (hibrit arama).



Şekil 2: Aynı TF-IDF sayıları için CSR saklama, dense bir diziden 54x daha az bellek istiyor (20K dokümanda 14 MB'a karşı 763 MB). Ters indeksli sparse·sparse kernel'imiz tek thread'li BLAS dense·dense'ten 81x, 768-d embedding'leri skorlamaktan 6x daha hızlı. Yine de dense embedding'leri (59 MB) seçiyoruz, çünkü doğru özeti daha sık birinci sıraya

koyuyorlar (hit@1 %74'e karşı %68,5) ve dokümanlarla anahtar kelime paylaşmayan yeniden ifade edilmiş sorguları da eşleştiriyorlar.

3.2 Boyut indirgeme

Normalize edilmiş 20.000 x 768 corpus üzerinde dört yöntem (`src/hpss/svd.py`): (1) **kendi blok power iteration'ımız**: tekil değer tahminleri 1e-6'dan az değişene kadar (en fazla 100 iterasyon) V ← qr(X^T (X V)) adımını tekrarlıyor ve X^T X'i hiç oluşturmuyor; (2) **NumPy tam SVD** (LAPACK, referans sonuç); (3) **scikit-learn TruncatedSVD** (randomize); (4) veriyi önce merkezleyen **scikit-learn PCA**.

```
V, _ = np.linalg.qr(rng.standard_normal((n, k)))   # random orthonormal start, n = 768
for it in range(max_iter):
    XV = X @ V                                      # m x k
    s_new = np.linalg.norm(XV, axis=0)              # singular value estimates
    if np.max(np.abs(s_new - s_old) / s_new) < tol: break
    s_old = s_new
    V, _ = np.linalg.qr(X.T @ XV)                   # amplify the top directions, keep them
orthonormal
```

Kod: `src/hpss/svd.py` – `power_iteration_svd` (L21–55); dört yöntem tek bir arayüzün arkasında, `reduce` (L62–86).

Uygulamamızın doğrulanması (k = 50, tam SVD'ye karşı; Şekil 3b, Ek D): tekil değerler medyan 1,1e-4 göreli hata içinde (en fazla 6,1e-3), altuzay örtüşmesi ≥ 0,993, yeniden oluşturma hatası optimumun yalnızca **%0,00013** üzerinde.

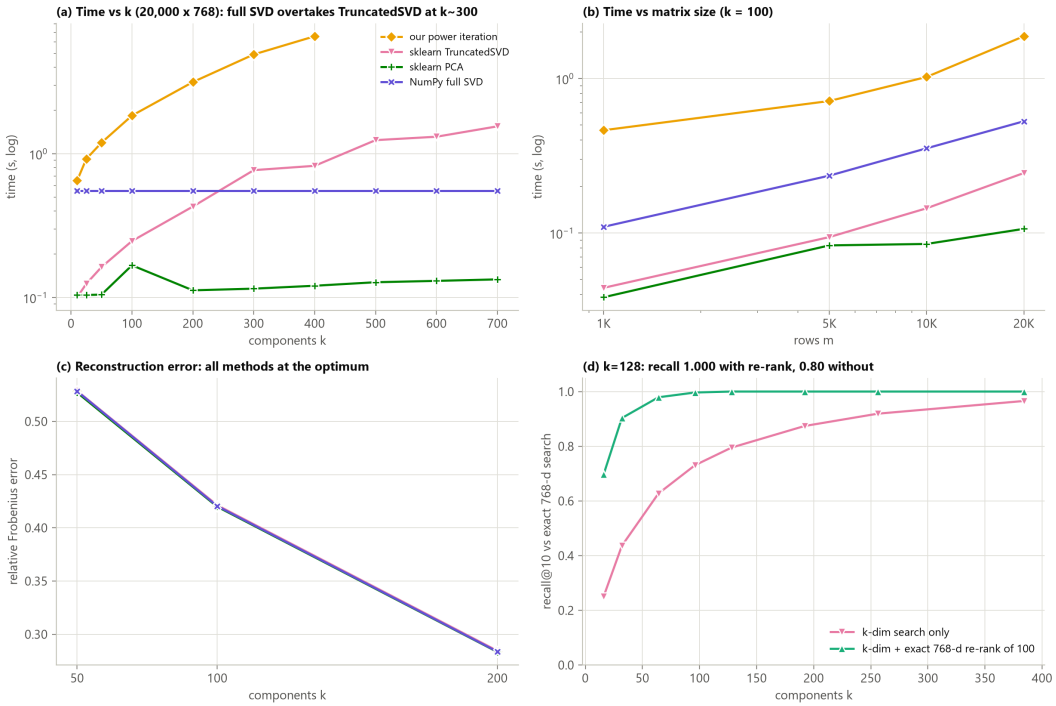
k	Yöntem	Süre	Göreli Frobenius hatası	Tepe bellek
50	power iteration (bizim) / tam SVD / TruncatedSVD / PCA	1,19 / 0,55 / 0,16 / 0,10 s	0,5283 / 0,5283 / 0,5286 / 0,5271	24 / 379 / 134 / 23 MB
100	power iteration (bizim) / tam SVD / TruncatedSVD / PCA	1,84 / 0,55 / 0,25 / 0,17 s	0,4204 / 0,4204 / 0,4212 / 0,4198	47 / 379 / 150 / 23 MB
200	power iteration (bizim) / tam SVD / TruncatedSVD / PCA	3,16 / 0,55 / 0,43 / 0,11 s	0,2837 / 0,2837 / 0,2844 / 0,2834	94 / 379 / 181 / 36 MB

Optimal hata (Eckart–Young) 0,5283 / 0,4204 / 0,2837. PCA'nın hatası bunun *altında* çıkıyor, çünkü ortalama vektörünü de saklıyor: fazladan bir serbestlik derecesi.

Truncated SVD tam SVD'den neden hızlı? Tam SVD, kaç k gerektiğinden bağımsız olarak 768 tekil vektörün hepsini hesaplıyor, O(m·n²) (sabit 0,55 s). Randomize çözücü X'i k + oversampling sayıda rastgele sütundan oluşan ince bir blokla birkaç kez çarpıyor, sonra küçük bir matrisin SVD'sini alıyor: O(m·n·k). Bu yüzden süresi k ile büyüyor (k = 10'da 0,10 s, k = 700'de 1,55 s) ve **k ≈ 300'den itibaren tam SVD kazanıyor** (Şekil 3a); bu, TruncatedSVD avantajının pratik üst sınırı. Power iteration'ımız iterasyon başına aynı O(m·n·k) işi yapıyor, ama oversampling olmadığı için ~100 iterasyon gerektiriyor: k = 100'de TruncatedSVD'den 7,5x yavaş.

PCA ne zaman SVD'den farklı? Verinin ortalaması sıfır olmadığında. Birim uzunluktaki embedding'lerimiz güçlü bir ortak yönü paylaşıyor: ortalama vektörün normu 0,52; merkezlenmemiş matrisin ilk tekil vektörü onunla %99,8 hizalı ve enerjinin %29'unu taşıyor. PCA ortalamayı ve onunla birlikte kosinüs aramasının dayandığı iç çarpımları çıkarıyor: aynı re-rank ile recall@10 = %98,1'e ulaşıyor (92 yanlış sıra), TruncatedSVD ve power iteration'ımız ise baseline sonucunu birebir döndürüyor.

Hangi k? İndirgenmiş uzay tek başına komşu kaybediyor (k = 128'de recall@10 %79,5, k = 384'te %96,6). Bu yüzden indirgenmiş uzayda 100 aday alıp onları kesin 768-d kosinüslerle re-rank ediyoruz; bu, k = 128'den itibaren %100'e ulaşıyor (k = 96'da %99,65). **k = 128 ile TruncatedSVD**'yi seçtik (fit ~0,3 s). **Dürüst bir olumsuz sonuç:** bu aşamada indirgeme aramayı *hızlandırmadı* – re-rank ile 22,0 ms, kesin 768-d aramada 15,4 ms. Çarpım küçüldü, ama 20.000 skor üzerindeki tek thread'li top-k ve re-rank olduğu gibi kaldı (Bölüm 6.1, bunun neden ancak top-k paralel olduktan sonra kazandırdığını gösteriyor). Hafta 5 eki olarak Dask'ın parçalı `svd_compressed`'ı (5.000 satırlık 4 parça, k = 128) aynı hatayla (0,3765'e karşı 0,3763; optimum 0,3754) TruncatedSVD'nin 0,29 s'sine karşı 0,22 s sürdü; üstelik out-of-core da çalışabilir.



Şekil 3: Truncated (randomize) SVD, k = 100'de optimal Eckart–Young hatasına (0,421 göreli) 0,25 s'de ulaşıyor; tam SVD 0,55 s, power iteration'ımız 1,84 s sürüyor. Tam SVD k ≈ 300'den itibaren onu geçiyor (a). k = 128'de, indirgenmiş uzaydaki 100 adayı tam 768-d'de re-rank etmek kesin top-10'un %100'ünü geri kazanıyor; re-rank olmadan bu oran %80 (d). PCA en hızlısı ve hatası en düşük olanı, ama recall'u yalnızca %98,1, çünkü merkezleme aramanın dayandığı kosinüs geometrisini değiştiriyor.

3.3 Vektörizasyon, eleman eleman ve JIT

Aynı işlemi, 200 x 20.000'lik kosinüs benzerlik matrisini, beş şekilde uyguladık (`src/hpss/kernels.py`). Beşi de aynı algoritmayı kullanıyor: satır başına bir kez norm, sonra çift başına bir nokta çarpımı. Hepsi tam iş yüküyle çalıştı:

Sürüm	Süre	Saf Python'a göre	GFLOP/s
Saf Python döngüleri (eleman eleman)	65,88 s	1x	0,09
NumPy: iki normalizasyon + bir BLAS matris çarpımı	26,0 ms	2.530x	236
Numba `@njit` (derlenmiş Python döngüleri)	1,44 s	46x	4,3
Numba `@njit(parallel=True)`, dokümanlar üzerinde `prange`	106 ms	623x	58
Numba parallel + `fastmath=True`	**19,0 ms**	**3.465x**	323

Kod: `src/hpss/kernels.py` – `cosine_matrix_python` (L28–42), `cosine_matrix_numpy` (L46–49), `cosine_matrix_numba` (L53–76), `cosine_matrix_numba_parallel` (L80–103), `cosine_matrix_numba_fastmath` (L107–130). Üç Numba sürümü aynı döngüler; yalnızca dekoratör (`@njit`, `parallel=True` + `prange`, `fastmath=True`) değişiyor.

Her adım neden daha hızlı? `@njit` (46x) yorumlayıcıyı kaldırıyor, ama döngü hâlâ her seferinde tek bir skaler çarpma-toplama yapıyor: katı IEEE sıralaması her 768 terimlik toplamı birbirine bağımlı toplamalardan oluşan bir zincire çeviriyor (~4 GFLOP/s). `prange` 16 thread'de 13,6x ekliyor. `fastmath` 5,6x daha ekliyor: LLVM'in toplamı yeniden sıralamasına ve birkaç bağımsız akümülatörle SIMD register'larını kullanmasına izin veriyor. NumPy çarpımı BLAS'a devrediyor (cache blocking, SIMD, thread'ler); 26 ms'sinin çoğu tek thread'de ayırdığı iki normalize kopyadan geliyor (yalnızca corpus için 13,4 ms) – çarpımın kendisi 4,3 ms (§6.1). **Derleme maliyeti:** yeni bir process'teki ilk derleme 0,46 s, sonrasında çağrı başına 1,1 ms; `cache=True` ile yeni bir process derlenmiş kodu diskten 0,26 s'de yüklüyor.

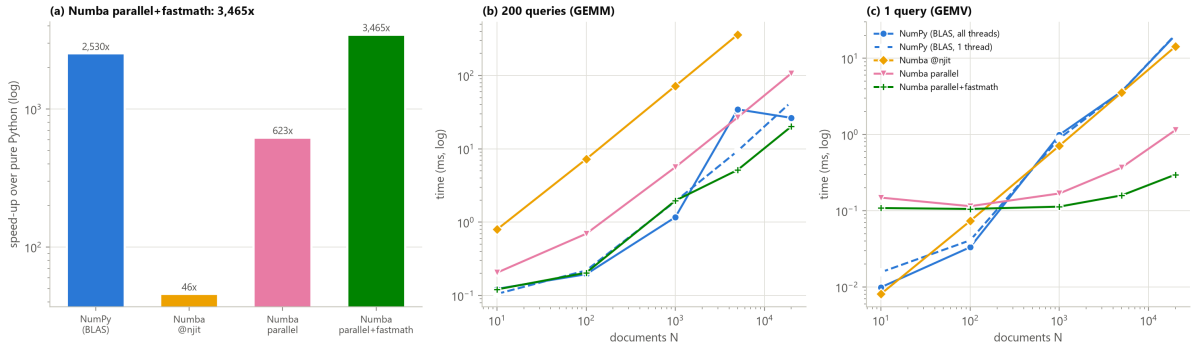
Numba NumPy'ı ne zaman yener, ne zaman kaybeder? (Şekil 4b/c)

- **Numba, NumPy'ın geçici dizilerle birkaç geçişte yaptığı işi tek kernel'de birleştirdiğinde kazanıyor.** 20.000 doküman üzerinde tek sorgu: NumPy 20,1 ms (neredeyse tamamı corpus'u tek thread'de yeni bir diziye normalize etmek), birleşik (fused) paralel Numba kernel'i **0,30 ms**. 200 sorguluk batch'te de Numba fastmath biraz önde (tabloda 19,0'a karşı 26,0 ms; Şekil 4b'nin ayrı boyut taramasında 20,2'ye karşı 26,5 ms).
- **NumPy küçük girdilerde ve düz dense çarpımlarda kazanıyor.** Numba'nın thread havuzunun ~0,1 ms'lik sabit bir başlatma maliyeti var (10 doküman: NumPy 10 µs, paralel Numba 0,11–0,15 ms) ve 1.000 dokümana kadar

NumPy daha hızlı ya da eşit (1,17'ye karşı 1,97 ms). Normalizasyon aramanın dışına taşındığında arama yalın bir matris çarpımına dönüşüyor ve BLAS'ın cache-blocked çarpımı (4,3 ms) yazabileceğimiz her döngüyü yeniyor.

- **Bir aykırı değer:** 5.000 dokümanda bütün BLAS thread'leriyle NumPy 34,5 ms, tek thread'le 9,4 ms sürdü; nedenini bulamadık (tek thread eğrisi düzgün).

Sonuç: matris çarpımı için BLAS, BLAS'ın yapamadığı işler – top-k seçimi ve re-rank – için Numba (Bölüm 6). **Bellek düzeni** (hafta 2, Ek D'deki Şekil 4d) aritmetik kadar önemliydi: fancy-index kopyası slice view'dan 3.012x, BLAS'sız bir `einsum Q @ X.T`'den 75x, re-rank öncesi aday satırlarını toplamak (gather) onları yerinde okumaktan 37x, float64 float32'den 2,07x yavaş; yeni bir diziye normalize etmek de yerinde yapmaktan 1,5x yavaş (ve iki kat tepe bellek). Satır/sütun öncelikli erişim yalnızca 1,04x fark ediyor, çünkü 59 MB'lık corpus 96 MB'lık L3'e sığıyor.



Şekil 4: Aynı döngüleri Numba ile derlemek saf Python'a göre 46x, `prange` 623x, `prange` + `fastmath` (SIMD) 3.465x kazandırıyor; NumPy/BLAS'ta bu 2.530x (a). Tek thread'li normalizasyon geçicileri baskın olduğunda NumPy kaybediyor: 20.000 doküman üzerinde tek sorgu için 20,1 ms gerekiyor, birleşik Numba kernel'i ise 0,30 ms (c). 200 sorguluk batch'te Numba fastmath biraz daha hızlı (20,2 ms'ye karşı 26,5 ms) (b).

4. Paralellik ve Eşzamanlılık

Referans noktası: top-k dahil en iyi tek thread'li Faz 2 araması: 1 BLAS thread'li NumPy, bir matris çarpımı + `argpartition`, **33,2 ms**. Ödevin **yedi stratejisinin hepsini** uyguladık (`src/hpss/parallel.py`); her biri tam 200 x 20K iş yükünde 1–16 worker ile ölçüldü ve baseline top-10'unu döndürdü.

Her stratejinin kodu (aksi belirtilmedikçe `src/hpss/parallel.py` içinde):

Strateji	Kod
Saf Python: thread / process (GIL)	`search_threadpool_python` (L91–100), `make_python_pool` + `search_processpool_python` (L111–132); free-threaded 3.14t: `scripts/gil_free_threading.py` (L22–78)
NumPy, BLAS thread'leri	`search_blas` (L51–53)
Numba `prange`	`search_numba_fused`, `src/hpss/kernels.py` (L188–217), `search_numba_threads` (L60–68)
`ThreadPoolExecutor` + NumPy	`search_threadpool` (L75–88)
`ProcessPoolExecutor` + paylaşımlı bellek / `memmap`	`SharedCorpusPool` (L151–189), `MemmapCorpusPool` (L196–229)
`asyncio`	`serve_serial`, `serve_concurrent`, `serve_batched` (L313–340)
Kuyruk + micro-batching	`serve_queue` (L233–276)
Dask	`search_dask` (L361–374), `search_dask_tree` (L385–394), `DaskDistributedSearch` (L409–429)
Ray	`ray_shard_task`, `ShardActor`, `RaySearch` (L436–512)
Shard birleştirme; yarış gösterisi	`merge_topk` (L39–44); `score_histogram_racy` / `_local`, `src/hpss/experiments/exp3_parallelism.py` (L181–202)

4.1 Saf Python: GIL (Şekil 5a)

Naif arama 48 sorgu x 1.000 dokümanda sıralı olarak 1,91 s sürüyor.

- **`ThreadPoolExecutor`: her thread sayısında 1,0x** (Amdahl p = 0,00). GIL aynı anda yalnızca bir thread'in bytecode çalıştırmasına izin veriyor (geçiş aralığı 5 ms).
- **`ProcessPoolExecutor`: 8 process'te 7,1x, 16'da 8,5x** (p = 0,95): her process'in kendi yorumlayıcısı ve GIL'i var. Ama process'ler veri taşımanın bedelini ödüyor: corpus'u her görevle göndermek toplamda 317 MB pickle'lıyor

(0,73 s), havuzun `initializer`'ı ile worker başına bir kez göndermek yalnızca 0,27 s; soğuk bir havuz (NumPy import eden 8 `spawn` edilmiş process) 2,04 s tutuyor, sıcak çalıştırmanın 7,7 katı.

- **Free-threaded CPython 3.14t** (PEP 703, GIL yok) aynı thread testini 2 / 4 / 8 / 16 thread'de 1,9x / 3,8x / 6,1x / 8,3x ölçekliyor, ama tek thread'i 1,85x daha yavaş (3,49'a karşı 1,89 s), dolayısıyla net kazanç 4,5x. GIL olmadan yarışlar gerçek oluyor: 1,6 milyon kez artırılan paylaşılan bir sayaç artışların %80'ini kaybediyor; her artışta kilit almak doğru ama 1,46 s sürüyor, thread başına yerel toplam ve thread başına tek kilitli ekleme ise 4 ms.

4.2 Vektörize arama: Amdahl yasası ve seri top-k (Şekil 5b)

Strateji	İşin bölünmesi	En iyi yapılandırma	Süre	Faz 2 en iyiye göre	Naife göre	Amdahl p
NumPy, BLAS thread'leri	matris çarpımının içinde	16 thread	16,0 ms	2,1x	9.631x	0,55
Numba `prange` (birleşik benzerlik + top-k)	sorgu blokları	16 thread	19,7 ms	1,7x	7.836x	0,98
`ThreadPoolExecutor` + NumPy (worker başına 1 BLAS thread)	sorgu parçaları	8 worker	10,1 ms	3,3x	15.304x	0,74
`ProcessPoolExecutor` + paylaşımlı bellek	sorgu parçaları	12 worker	10,7 ms	3,1x	14.369x	0,74
`ProcessPoolExecutor` + `np.memmap` dosyası	sorgu parçaları	8 worker	9,7 ms	3,4x	15.866x	–
Dask `delayed` (thread'ler)	8 doküman shard'ı + birleştirme	8 worker	**8,9 ms**	**3,7x**	17.390x	0,79
Ray actor'ları (durumlu shard'lar)	8 doküman shard'ı	8 shard	10,6 ms	3,1x	14.576x	–
Ray görevleri (`ray.put` shard'ları)	8 doküman shard'ı	8 shard	11,8 ms	2,8x	13.049x	–

BLAS thread'leri 2,1x'te duruyor ve nedeni Amdahl yasası. İki adımı ayrı ayrı ölçtük:

BLAS thread'i	Matris çarpımı	Top-k (`argpartition`)	Seri pay
1	22,8 ms	10,1 ms	%31
16	4,3 ms	10,2 ms	**%70**

Çarpım daha fazla thread'le 5,3x hızlanıyor, ama tek thread'li top-k değişmiyor: çarpım paralelleşince top-k sürenin %70'i oluyor. Amdahl yasası speed-up'ı $1/(1 - p)$ ile sınırlar: tek thread'de ölçülen %31'lik seri payla ($p = 0,69$) tavan 3,2x; fit edilen $p = 0,55$ daha da düşük, çünkü çarpımın kendisi de 16 thread'de yalnızca 5,3x hızlanıyor (ortak bellek bant genişliği, SMT). **Sorgulara ya da shard'lara bölmek top-k'yı da paralelleştiriyor** ($p = 0,74$–$0,79$). Yine de doğrusallıktan uzak kalıyorlar, çünkü her worker 58,6 MB'lık corpus'un tamamını aynı L3 ve bellek yolu üzerinden akıtıyor (bant genişliği), 25 sorguluk bir arama çağrısı sorgu başına 0,28 ms tutarken 200 sorgunun tamamında 0,17 ms tutuyor (tek BLAS thread'i) ve 8 worker'ın üzerinde iki SMT thread'i bir çekirdeğin kayan nokta birimlerini paylaşıyor. **Birleşik Numba kernel'i neredeyse kusursuz ölçekleniyor** (8 çekirdekte 8,0x, 16'da 11,6x, $p = 0,98$: seri kısım yok, yalnızca salt-okunur paylaşım), ama yavaş bir tek thread'den (228 ms) başlıyor, çünkü cache blocking olmadan corpus'u her sorgu için bir kez baştan sona okuyor. **İyi ölçeklenmek, hızlı olmak demek değil.**

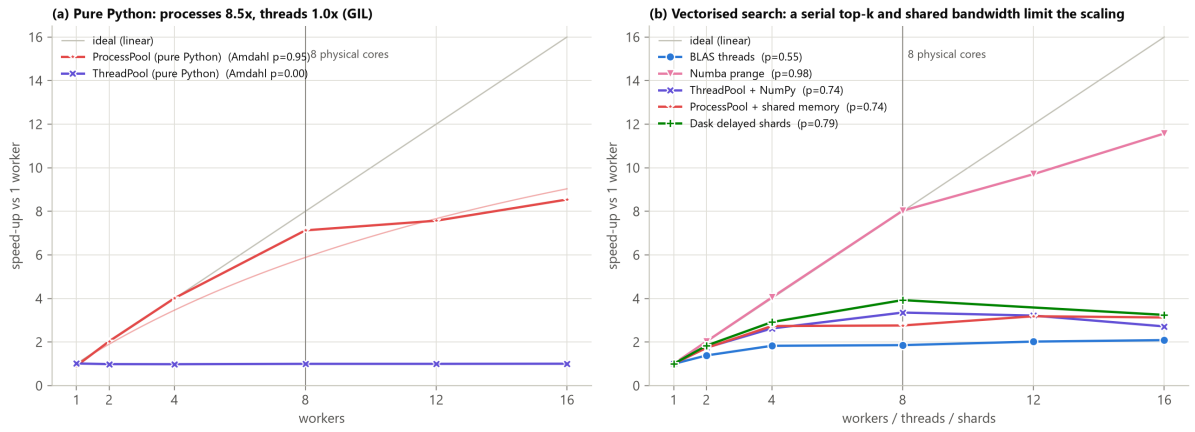
Ölçtüğümüz üç tuzak. *Oversubscription:* worker'larının her biri 16 BLAS thread'i kullanan bir thread havuzu (16 donanım thread'inde 256 thread) 12,4 ms yerine 84,9 ms sürüyor. *Granülerlik* (Şekil 5c, Ek D): en iyi görev boyutu 25 sorgu (8 worker için 8 görev, 10,0 ms); 2 sorguluk görevler (46,6 ms) 1 sorguluk görevlerden (17,6 ms) bile yavaş, çünkü iki sorguda BLAS matris-vektörden matris-matrise geçiyor ve bu da önce bütün corpus'u paketliyor (çağrı başına ~3,2 ms). *`prange` içinde yarış:* paylaşılan tek bir histograma ekleme yapan 16 thread, 4 milyon güncellemenin 2,6 milyonuna kadarını kaybediyor ve en sonda bir kez birleştirilen özel satırlardan 50x daha yavaş (17,0'a karşı 0,34 ms), çünkü çekirdekler aynı cache satırları için çekişiyor.

4.3 Hangi araç ne için

Araç	Ne ölçtük	Şu durumda doğru araç …
Process havuzu + paylaşımlı bellek / `memmap`	Corpus bir kez yerleştiriliyor; bir görev 75 KB sorgu gönderip 2,9 KB geri alıyor: 10,7 / 9,7 ms. Soğuk havuz 2 s tutuyor.	CPU-bound Python; birçok process'in paylaştığı salt-okunur veri
`asyncio`	Simüle edilmiş 20 ms'lik bir API'yi bekleyen 100 sorgu: sıralı 2,85 s, `gather` 41,9 ms (68x), `gather` + tek batch'li çarpım 40,6 ms. CPU işinde kazanç yok (18,7'ye karşı 19,8 ms).	Çok sayıda eşzamanlı I/O beklemesi (embedding API çağrıları)
Kuyruk + micro-batching	200 sorgu aynı anda: 200'lük batch'ler 12.453 q/s (p95 16 ms), 1'lik batch'ler 5.417 q/s (p95 35 ms). Her 0,5 ms'de bir sorgu: her batch boyutunda ~986 q/s.	Çevrimiçi servis, kuyruk oluştuğunda
Dask	Thread'ler 8,9 ms (birleştirme = sürenin %4'ü, Şekil 5d); process'ler çağrı başına 1,04 s (shard'ları gönderiyor); `LocalCluster` 38 ms + 1,6 s başlatma; `dask.array` 48–58 ms; 64 shard'da düz birleştirme 20,0 ms, ağaç 22,7 ms.	RAM'den büyük veri; çok sayıda makine
Ray	`ray.init` 12,1 s; actor'lar 10,6 ms, görevler 11,8 ms, `ray.wait` 11,6 ms; denemelerin %30'u başarısız olurken bile kesin (2,0 s, yeniden denemeler).	Çok sayıda bağımsız görev, durumlu worker'lar, hata toleransı

Saf Python bir hat (hafta 5: 3,56 M kelime üzerinde doküman frekansları) da aynı örüntüyü gösteriyor: sıralı `Counter` 0,41 s, thread'li Dask Bag 0,64 s (GIL), process'li Dask Bag 0,68–2,17 s, ağaç birleştirmeli Ray 5,6 s'lik başlatmanın ardından 0,26 s. Windows'ta `asyncio.sleep(0.02)` gerçekte 29,5 ms uyuyor (zamanlayıcı çözünürlüğü).

Kod karmaşıklığı başına en iyi speed-up? BLAS thread'leri sıfır satıra mal oluyor, ama seri top-k yüzünden 2,1x'te kalıyor. Faz 3 stratejileri arasında `ThreadPoolExecutor` + NumPy ~15 satırla 3,3x, Dask `delayed` (thread'ler) benzer kodla en iyi sonucu, 3,7x'i veriyor; process havuzları ~60 satır ve bir başlatma maliyeti istiyor ama daha fazlasını vermiyor, Ray ise 12 s'lik bir başlatma istiyor. **Genelde en iyi oran, seri kısmı ortadan kaldıran ~20 satırlık bir Numba top-k** (§6.1): kesin arama 15,5 → 5,4 ms, her framework'ten hızlı. **10 kat büyük bir corpus için** strateji aynı kalıyor – 200K doküman (586 MB) hâlâ RAM'e sığıyor, dolayısıyla tek process içindeki thread'ler doğru seçim olmaya devam ediyor – ama darboğaz bellek bant genişliğine dönüşüyor (corpus L3'ten çıkıyor), bu yüzden batching ve daha küçük 128-d indeks daha önemli hâle geliyor (§6.3). Dask ve Ray ancak veri tek bir makineye sığmadığında doğru seçim oluyor (§6.5).



Şekil 5: Saf Python'da process'ler 8,5x'e ulaşıyor (Amdahl p = 0,95), thread'ler ise GIL yüzünden 1,0x'te kalıyor (a). Vektörize aramada BLAS thread'leri 2,1x'te duruyor (p = 0,55), çünkü matris çarpımından sonraki top-k seçimi tek thread'li. İşi sorgu parçalarına ya da shard'lara bölen stratejiler seçimi de paralelleştiriyor (p = 0,74–0,79) ve birleşik Numba kernel'inin hiç seri kısmı yok (p = 0,98) (b).

5. Tasarım Kararları

Karar	Seçim	Değerlendirilen alternatifler	Kanıt
Temsil	Dense 768-d embedding'ler	CSR'de TF-IDF (54x daha az bellek, 6x daha hızlı)	Embedding'ler doğru özeti daha sık buluyor (hit@1 %74'e karşı %68,5) ve yeniden ifadeleri yakalıyor (§3.1).
Hassasiyet	float32	float64	Matris çarpımı 2,07x daha hızlı. En büyük skor hatası 4,9e-7, 1e-5 toleransının çok içinde (beklenen hata yaklaşık √768 · 1,2e-7 ≈ 3,3e-6) (§3.3, §6.2).
Normalizasyon	Bir kez, indeksleme sırasında, yerinde	Her aramada; yeni diziler	Python: 2,3x (§2.5). NumPy: corpus'u her çağrıda normalize etmek 13,4 ms tutuyor, 4,3 ms'lik çarpımdan fazla (§3.3).
Boyut indirgeme	TruncatedSVD, k = 128	Tam SVD; power iteration'ımız; PCA	Optimal hatayla ~0,3 s'de fit. Tam SVD k ≈ 300'ün altında daha yavaş; power iteration 7,5x yavaş; PCA kosinüs geometrisini bozuyor (%98,1 recall) (§3.2).
Re-rank için aday	c = 100	10, 20, 50, 200	200 sorgunun hepsinde, bütün boyutlarda 0 sıra ihlali veren en küçük c (§6.2).
Benzerlik kernel'i	Tek batch'li BLAS matris çarpımı (GEMM)	Birleşik Numba kernel'i; sorgu başına bir matris-vektör çarpımı	Batch'li GEMM hesap sınırlı çalışıyor (1,1–1,4 TFLOP/s, Ek C). Birleşik kernel, corpus L3'ten çıkınca 10x veri için 36x yavaşlıyor (§6.3).
Top-k seçimi	Paralel Numba top-k	`np.argpartition` (1 thread); tam sıralama	0,24 ms'ye karşı 10,2 ms (43x). Bütün paralel stratejileri sınırlayan seri pay buydu (§4.2, §6.1).
Paralellik	Tek process içinde thread'ler (BLAS + Numba)	Process havuzları, Dask process / dağıtık, Ray	Veri RAM'e ve L3'e sığıyor; process havuzları 1–2 s başlatma ekliyor, Ray 12 s. Bu boyutta seri kısmı kaldırmak (5,4 ms) her framework'ü (8,9–11,8 ms) yeniyor (§4).
İki thread havuzu	`OMP_WAIT_POLICY=PASSIVE` (`import hpss` ayarlıyor)	Varsayılan, dönen (spinning) OpenMP thread'leri	Dönen Numba thread'leri sonraki BLAS çarpımını 4,4'ten 34,1 ms'ye yavaşlatıyor (§6.1).
Batching	Çağrı başına 200 sorgu (GEMM)	Çağrı başına bir sorgu (GEMV)	Sorgu başına 0,17 ms'ye karşı 0,53 ms (arama çağrısı, 1 BLAS thread'i); son motorda tek bir sorgu yalnızca 0,21 ms (§4.2, §6.3).

6. Son Sistem ve Ölçekleme

6.1 Kendi motorumuzu profillemek: seri kısım yer değiştirdi

Faz 3'ten sonra kendi aramamızı, baseline'ı profillediğimiz gibi aşama aşama profilledik (200 x 20K, ms cinsinden):

Motor varyantı	Normalize / izdüşüm	Matris çarpımı	Top-k	Re-rank	Toplam
Kesin 768-d + `argpartition`	0,09	4,28	**10,23**	–	15,55
Kesin 768-d + paralel top-k	0,09	4,32	0,24	–	5,36
SVD-128 + re-rank(100) + `argpartition` + gather/ einsum	0,36	1,86	**10,12**	**8,84**	21,93
SVD-128 + re-rank(100) + paralel top-k + yerinde re-rank	0,20	1,82	0,58	0,45	**3,67**

Ayrı deneyler son motor için 3,64–3,89 ms veriyor; manşet sayılar ölçekleme koşusundan (3,89 ms, §6.3).

- **Yeni darboğaz top-k:** tek thread'li `argpartition` kesin aramanın %66'sı. Paralel bir Numba top-k (her thread kendi sorgu bloğu için 10 elemanlı sıralı bir tampon tutuyor) o adımı 43x hızlandırıyor.
- **Bu, §3.2'deki olumsuz sonucu açıklıyor.** 128-d çarpım 768-d olandan 2,3x ucuz, ama tasarruf ancak top-k (10,1 → 0,58 ms) ve re-rank (8,8 → 0,45 ms: aday satırlarını bir kopyada toplamak yerine yerinde okumak) düzeltildikten sonra görünüyor.
- **İki thread havuzu birbiriyle yarışmamalı.** Numba'nın OpenMP thread'leri varsayılan olarak paralel bölgeden sonra dönmeye (spin) devam ediyor ve sonraki BLAS çarpımı çekirdekler için onlarla çekişiyor: **4,4 ms yerine 34,1 ms**. `OMP_WAIT_POLICY=PASSIVE` ile boştaki thread'ler uyuyor ve son motor 7,01 yerine 3,64 ms sürüyor (Şekil 6a, Ek D).

6.2 Son hat ve doğruluğu

```
class SearchEngine:                       # src/hpss/search.py (abridged)
    def fit(self, X):                     # once, at indexing time
        self.X  = normalize_rows(X).astype(np.float32)                  # 20,000 x 768, unit rows, C
order
        fit     = reduce(self.X, 128, method="truncated_svd")           # ~0.3 s
        self.P  = fit["components"].astype(np.float32)                  # 128 x 768
        self.Xr = normalize_rows(self.X @ self.P.T)                     # 20,000 x 128 index (9.8
MB)

    def search(self, Q, k=10):
        Qn   = normalize_rows(Q)                                        # 200 x 768
        S    = normalize_rows(Qn @ self.P.T) @ self.Xr.T                # BLAS GEMM in 128-d: 1.8 ms
        cand, _ = topk_rows(S, 100)                                     # parallel Numba top-100:
0.6 ms
        exact   = candidate_scores(Qn, self.X, cand)                    # exact 768-d cosines, in
place: 0.5 ms
        order, scores = topk_from_scores(exact, k)                      # top-10 of 100
        return np.take_along_axis(cand, order, axis=1), scores
```

Kod: `src/hpss/search.py` içinde `SearchEngine` (L53–133); paralel top-k `topk_rows` ve yerinde re-rank `candidate_scores`, `src/hpss/kernels.py` içinde (L137–181).

Hat: veri → embedding (çevrimdışı) → normalize → TruncatedSVD ile 128-d → batch'li BLAS araması → paralel top-100 → kesin 768-d re-rank → top-10. Re-rank, seçilen adayların skorlarını tam 768-d uzayda hesaplıyor; aday havuzuna alınmayan gerçek komşuları geri getirmiyor. Dolayısıyla bu hat yaklaşık aramadır. Aşağıdaki %100 recall sonucu ölçülen 200 sorguya aittir; görülmemiş sorgular için kesinlik garantisi değildir.

Kaç aday gerekiyor (200 x 20K)?

Aday sayısı c	Süre	recall@10	Sıra ihlali	En büyük skor farkı
10	2,92 ms	0,795	749	0,075
50	3,20 ms	0,994	33	0,011
100	**3,70 ms**	**1,000**	**0**	**7,7e-8**
200	4,93 ms	1,000	0	7,7e-8

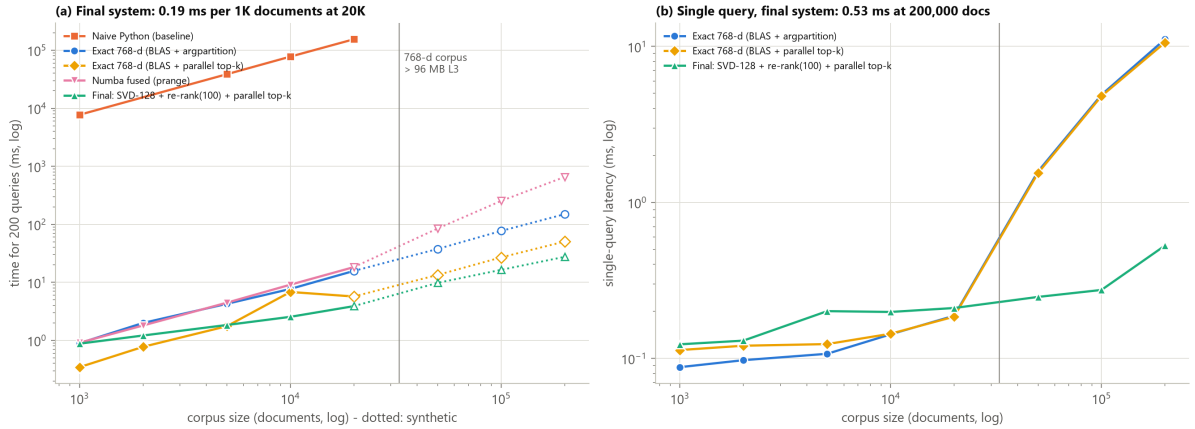
Doğruluk: dört sürüm (kesin BLAS, kesin + paralel top-k, birleşik Numba, son motor) 1K, 5K, 10K ve 20K dokümanda her sorgu için baseline'ın top-10'unu döndürüyor (indeks eşleşmesi %100, eşit skorlarda yer değiştirme yok). En büyük skor farkı kesin float32 BLAS için 4,9e-7, son motor için 7,7e-8; ikisi de 1e-5 toleransının çok altında.

6.3 Ölçekleme (Şekil 6)

1K, 2K, 5K, 10K ve 20K gerçek dokümanda ve – veri setinin ötesini görmek için – 50K, 100K ve 200K sentetik dokümanda (gerçek embedding'lerin küçük gürültü eklenmiş kopyaları; ekstrapolasyon değil, ölçüm) arama süresi. Tablo bir alt küme; bütün boyutlar Şekil 6'da ve `results/4_final_system/scaling.csv` dosyasında.

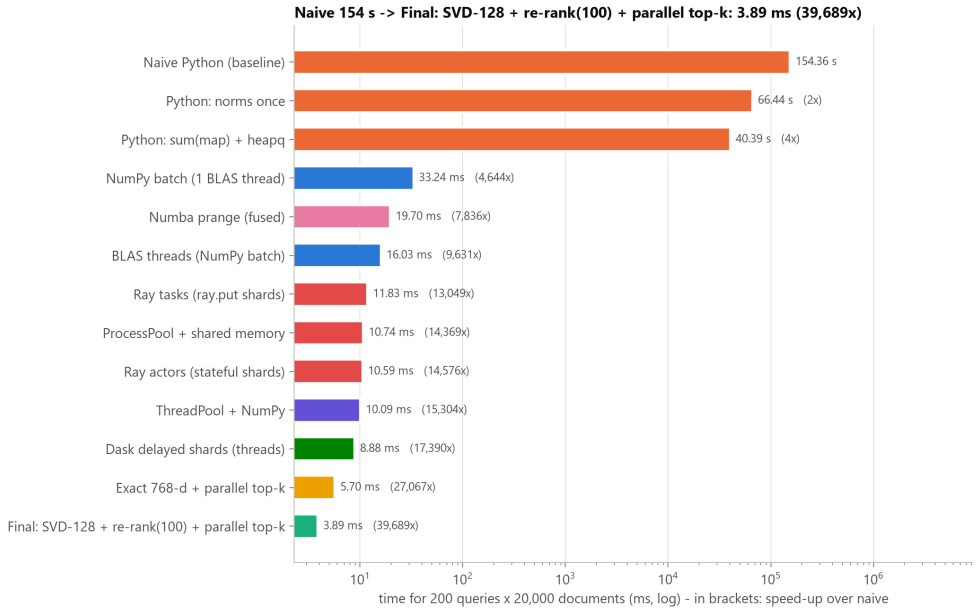
200 sorgu	1K	5K	10K	20K	200K (sentetik)	Eğim 5K–20K	Eğim 20K–200K
Naif Python	7,73 s	38,6 s	77,6 s	154,4 s	–	1,00	–
Kesin BLAS + `argpartition`	0,89 ms	4,27 ms	7,70 ms	15,7 ms	149 ms	0,94	0,98
Birleşik Numba	0,91 ms	4,46 ms	9,11 ms	18,3 ms	655 ms	1,02	**1,56**
Son motor	0,88 ms	1,84 ms	2,55 ms	**3,89 ms**	27,5 ms	**0,54**	0,85

Sistem doğrusal ölçekleniyor mu? En fazla doğrusal: son motor 5K ile 20K doküman arasında 0,54'lük log-log eğimiyle büyüyor (20K'da 1.000 doküman başına 0,19 ms). Alt-doğrusal, çünkü birkaç maliyet N'ye bağlı değil – sorguların izdüşümü, thread'lerin başlatılması, sorgu başına 100 adayın seçilip re-rank edilmesi. **Nerede bozuluyor? Corpus 96 MB'lık L3 cache'ten çıktığında.** Kesin tek sorgu gecikmesi 20K'da (59 MB) 0,19 ms, ama 50K'da (147 MB) 1,58 ms – 2,5x veri için 8,4x süre – ve 200K'da 11,1 ms. Corpus'u her sorgu için bir kez okuyan birleşik Numba kernel'i en çok etkileniyor (10x veri için 36x yavaş). Son motor 128-d indeksi (200K'da 98 MB) tarıyor ve sorgu başına 768-d matrisin yalnızca 100 satırına dokunuyor: 200K'da tek sorgu başına **0,53 ms**. **10 kat daha fazla veriyle** tam olarak bu motoru korurduk – hâlâ RAM'e sığıyor ve indirgenmiş indeks taramayı cache'e yakın tutuyor (200K'da 200 sorgu 27,5 ms).



Şekil 6: Arama süresi corpus boyutuyla en fazla doğrusal büyüyor. Son motorun log-log eğimi 5K–20K doküman arasında 0,54, 200K sentetik dokümana kadar 0,85; 1'in altında, çünkü çağrı başına sabit maliyetler hâlâ önemli (a). Corpus'u her sorgu için yeniden okuyan birleşik Numba kernel'i, corpus 96 MB'lık L3 cache'ten çıkınca 10x veri için 36x yavaşlıyor; kesin tek sorgu gecikmesi de öyle (b). Son motor 20K doküman üzerinde 200 sorguyu 3,89 ms'de, tek bir sorguyu 0,21 ms'de cevaplıyor.

6.4 Bütün sürümler



Şekil 7: Aynı 200 x 20K iş yükünde, özdeş sonuçlarla, 154 s'den (naif Python) 3,89 ms'ye (son motor: SVD-128 + re-rank(100) + paralel top-k): 39.689x speed-up. Yalnızca Python düzeyindeki düzeltmeler en fazla 3,8x'e ulaşıyor; büyük

sıçrama yorumlayıcıdan çıkmaktan (BLAS / Numba, 4.644x), son 8,5x ise kendi motorumuzu profilleyip seri kısmını kaldırmaktan geliyor.

Canlı demo: `python scripts/demo.py --compare-naive "speculative decoding for LLM inference"` indeksi yüklüyor ve yazılan sorguları gerçek zamanlı olarak, istenirse naif aramanın yanında cevaplıyor.

6.5 100 milyon dokümanda ne değişirdi (hafta 5)

100M dokümanda corpus artık tek bir makineye sığmıyor (768-d'de 307 GB, 128-d'de 51 GB; RAM ise 33,5 GB = 31,2 GiB) ve 614 MB'lık bir matris-vektör taramasında ölçtüğümüz 58,9 GB/s ile tek bir tam tarama 5,2 s sürüyor. Motor **dokümana göre shard'lanır** (makine başına bir shard; veri değil sorgu taşınır – shard başına 3,1 KB gider, 120 B döner), tam tarama yerine yaklaşık bir indeks (HNSW/IVF, Ek C) kullanır ve başarısız shard görevlerini yeniden dener (1.000 makine x 24 saatte bir yerde arıza çıkma olasılığı %63). Pahalı adım arama değil, encoding (embedding üretimi) olur: ölçtüğümüz saniyede 9,1 özetle 100M özet 127 gün. Bu duvarların arkasındaki sayılar Ek E'de.

7. Tartışma ve Dersler

Bizi şaşırtanlar:

1. **İlk yanılan kod değil, profiler oldu.** Python 3.12'de Jupyter içindeki cProfile, süreyi kernel'in I/O thread'lerine yazdı (bir `_select` satırı çıktı ve `list.sort` onun tarafından çağrılmış gibi gösterildi): 3.12'nin `sys.monitoring`'i bütün thread'leri görüyor ama tek bir çağrı yığını tutuyor. Ayrı bir process'te profillemek sorunu çözdü. Ders: bir profilin makul olup olmadığını kontrol edin.
2. **Boyut indirgeme tek başına aramayı yavaşlattı** (22,0'a karşı 15,4 ms): maliyet, indirgemenin küçülttüğü çarpımda değil, ancak kendi motorumuzu profillemenin (§6.1) gösterdiği top-k ve re-rank'teydi. En çok önemli olan optimizasyon en az gösterişli olanıydı: 20 satırlık bir paralel top-k.
3. **Doğru ve hızlı iki thread havuzu birbirini 8x yavaşlatabiliyor.** Dönen OpenMP thread'leri aynı çekirdekler için BLAS thread'leriyle çekişti (34,1'e karşı 4,4 ms).
4. **Büyük framework'ler bu ölçekte kaybetti.** Ray yalnızca başlamak için 12 s, Dask'ın process zamanlayıcısı çağrı başına 1 s istiyor. 58,6 MB'lık corpus L3 cache'e sığıyor, dolayısıyla sade thread'ler kazanıyor.
5. **Free-threaded Python saf Python thread'lerini gerçekten ölçekliyor** (16'da 8,3x), ama tek thread 1,85x daha yavaş ve GIL'in eskiden gizlediği yarışlar hemen ortaya çıkıyor.

Farklı ya da daha fazla zamanla ne yapardık? Bütün motoru Faz 3'ten sonra değil, Faz 2C'nin hemen ardından aşama aşama profillerdik – seri top-k'yı bir faz önce bulurduk. Corpus'u int8 ya da product quantization ile saklardık (4–32x daha az bellek trafiği), ~1M dokümanın ötesinde bir IVF ya da HNSW indeksi kullanırdık, GPU sürümünü çalıştırırdık (hafta 6, burada kapsam dışı) ve 96 MB'lık L3 gibi makineye özgü etkileri genel olanlardan ayırmak için ölçekleme testini ikinci bir makinede (ör. Colab) tekrarlardık.

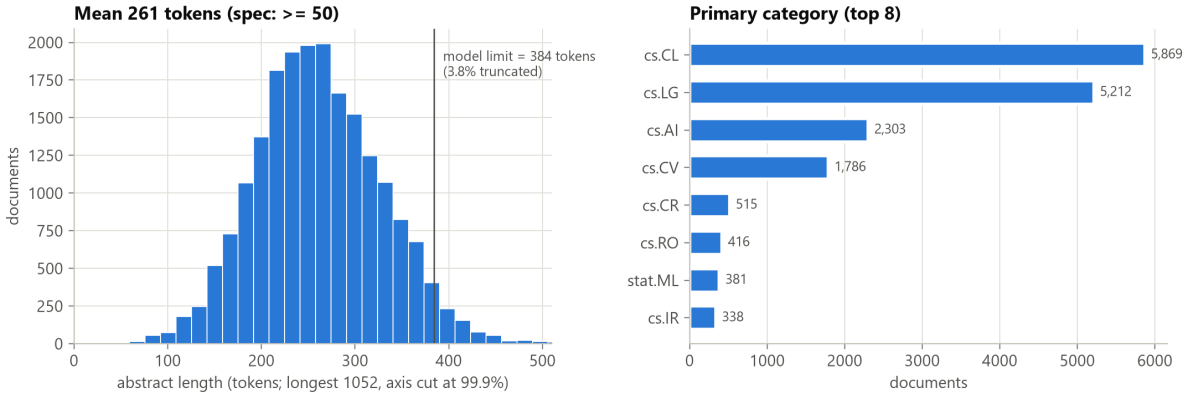
Gerçek yapay zekâ sistemleriyle bağlantı. Vektör veritabanları (FAISS, Milvus, pgvector) ve RAG sistemlerinin geri getirme adımı, ölçtüğümüz parçalardan kurulu: normalize float32 vektörler ve batch'li matris çarpımları (CPU'da BLAS, GPU'da cuBLAS), heap ya da partition tabanlı top-k, iki aşamalı geri getirme (ucuz aday üretimi, ardından kesin ya da cross-encoder re-rank) ve shard başına top-k listelerinin birleştirildiği, dokümana göre shard'lama. Sayılarımız bu tasarımların neden var olduğunu gösteriyor: aritmetik ucuz; zamanı bellek trafiği, seri adımlar ve veri taşımak harcıyor.

Sınırlamalar.

- **Tek makine**, alışılmadık derecede büyük L3 cache'li bir Windows masaüstü. 16–32 MB L3 ile (tipik bir dizüstü) cache uçurumunun 20K ile 50K arasında değil, ~5–10K dokümanda görülmesi beklenir.
- **20K'nın ötesinde sentetik veri:** gerçek embedding'lerin gürültülü kopyaları – süre için geçerli, kalite için değil.
- **c = 100 aday** bu 200 sorguda, dört corpus boyutunda doğrulandı; görülmemiş sorgular için garanti değil (gerçek komşuları 128-d'de iyi temsil edilmeyen bir sorgu birini kaçırabilir).
- **Milisaniye ölçeğindeki süreler işletim sisteminin zamanlamasına duyarlı.** ≥ 5 çalıştırmanın medyanını raporluyoruz (std `results/` altında); birkaç nokta gürültülü (10K'da kesin arama: std 8–14 ms) ve bir aykırı değer açıklanamadı (5K'da NumPy, §3.3).
- **Sorgular makale başlıkları**, yani belirli bir sorgu türü; embedding süresi (corpus için 2.199 s), ödevin belirttiği gibi her benchmark'ın dışında.

Ek

A. Veri seti ayrıntıları



Şekil 0: Corpus, ortalama 261 token uzunluğunda 20.000 arXiv özetinden (2025–26) oluşuyor; bu, 50 token'lık alt sınırın çok üzerinde. Yalnızca %3,8'i embedding modelinin 384 token'lık penceresini aşıyor.

Ham kayıtlar, her biri 500 makalelik 63 (ay, kategori) hücresinden geliyor. Ön işleme `src/hpss/data.py` içinde; işlenmiş corpus ve sorgular `data/processed/` altında (`corpus.jsonl`, `queries.jsonl`).

B. Kod, gereksinimler ve yeniden üretim

- **Depo:** https://github.com/ahmetozanyildirim/yzm513-similarity-search. Kodun tamamı `src/hpss/` içinde; her faz için bir deney modülü `src/hpss/experiments/` içinde.
- **Gereksinimler:** `requirements.txt`. Windows 11'de Python 3.12.10 ile test edildi. Ana paketler: numpy, scipy, scikit-learn, pandas, matplotlib, threadpoolctl, psutil, sentence-transformers, line_profiler, py-spy, numba, dask[array,distributed], ray ve faiss-cpu.
- **Yeniden üretim:**

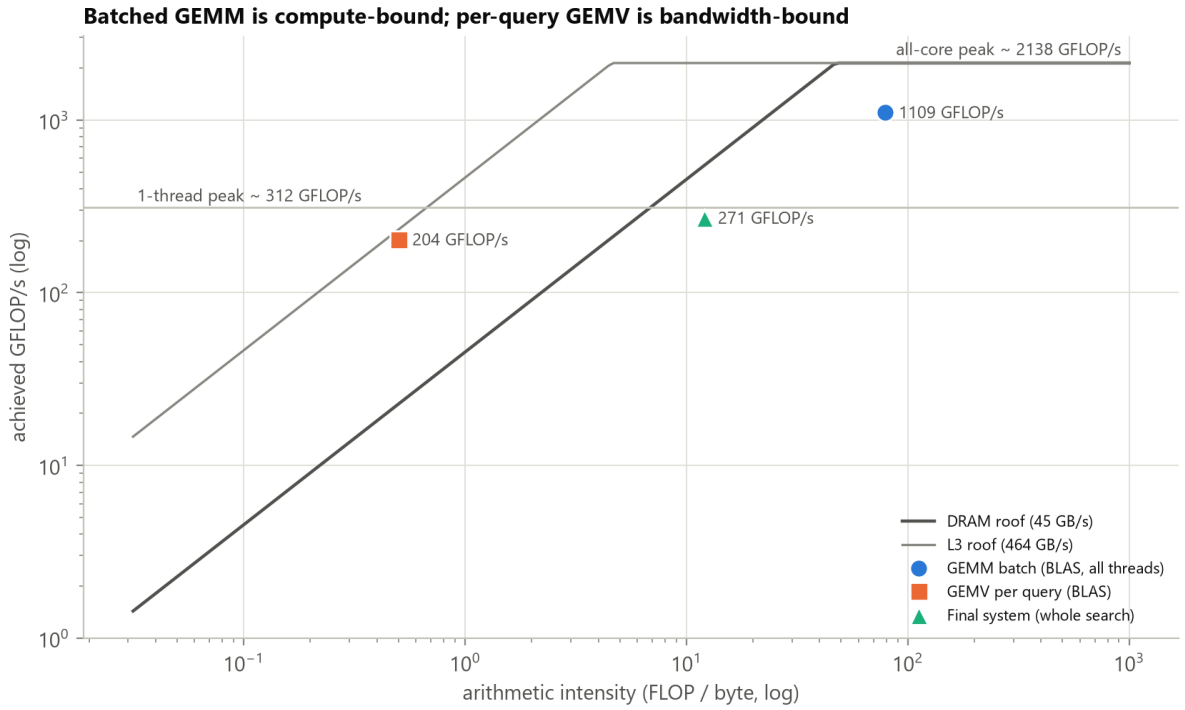
```
python -m venv .venv && .venv\Scripts\activate      # Linux/macOS: source .venv/bin/activate
pip install -r requirements.txt && pip install -e .
python scripts/prepare_data.py      # download + preprocess + embed (cached)
python scripts/run_all.py           # all phases, ~70 min on this machine (full workloads, no
extrapolation)
python scripts/build_report.py      # this PDF (Markdown -> HTML -> PDF with Edge/Chrome)
python scripts/demo.py --compare-naive "speculative decoding for LLM inference"
```

- **Ham benchmark verisi:** her faz `results/<phase>/results.json` (ana sayılar ve şekil açıklamaları), her ölçümü içeren CSV dosyaları (medyan, std, gerektiğinde ham çalıştırmalar) ve her şekli PNG ve PDF olarak yazıyor.
  - **Faz 1:** `naive_scaling.csv`, `python_fixes.csv`, `cprofile.txt`, `line_profiler.txt`, `flamegraph.svg`, `baseline_topk.npz`.
  - **Faz 2A:** `times.csv`, `memory.csv`, `density_sweep.csv`.
  - **Faz 2B:** `time_vs_k.csv`, `time_vs_m.csv`, `errors_memory.csv`, `search_quality.csv`.
  - **Faz 2C:** `main_comparison.csv`, `size_sweep.csv`, `memory_layout.csv`.
  - **Faz 3:** `scaling_all.csv`, `summary.csv` ve deney başına bir CSV (A–I).
  - **Faz 4:** `scaling.csv`, `correctness.csv`, `candidates.csv`, `stage_profile.csv`, `thread_contention.csv`, `scale_*.csv`.
  - **Faz 5:** `roofline_points.csv`, `ann.csv`.

C. Bonus: roofline modeli ve yaklaşık en yakın komşular

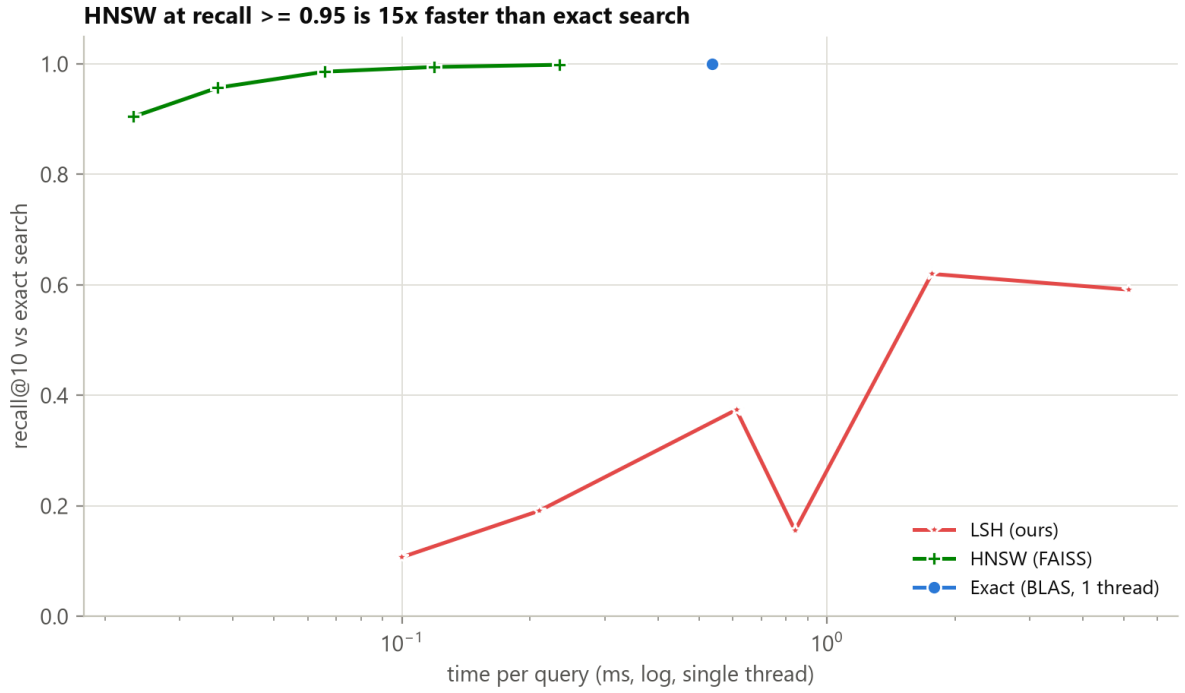
Bu raporda kapsanan bonus maddeler:

- **3'ten fazla paralellik stratejisi:** 7'sinin tamamı (§4).
- **Hem Dask hem Ray** (§4.3).
- Aşağıda **bir roofline modeli** ve **yaklaşık en yakın komşular**.



Şekil 8 (bonus): Batch'li matris çarpımı 1.109 GFLOP/s'de çalışıyor (aritmetik yoğunluk ≈ 79 FLOP/B, ölçülen 2.138 GFLOP/s'lik hesap tavanının %52'si). Sorgu başına matris-vektör çarpımı AI = 0,5'te yalnızca 204 GFLOP/s'ye ulaşıyor: L3 bant genişliğinin (464 GB/s) bu yoğunlukta izin verdiği 232 GFLOP/s'nin %88'i. Aramayı bellek sınırlıdan hesap sınırlıya taşıyan şey sorguları batch'lemek.

Roofline girdileri: ölçülen hesap tavanı bütün çekirdeklerde 2.138 GFLOP/s, tek thread'de 312 GFLOP/s. Ölçülen bant genişliği L3'ten 464 GB/s, DRAM'den 45 GB/s. Son motorun tamamı 271 GFLOP/s'de çalışıyor (AI ≈ 12): ucuz 128-d çarpımı artık baskın maliyet değil. Roofline'daki GEMM noktası ayrı ölçüldü (200 x 20K için 5,5 ms); motor profilinde aynı çarpım 4,3 ms sürdü (≈ 1.430 GFLOP/s). DRAM tavanı 3 GB'lık bir STREAM triad'ından (eleman başına iki okuma, bir yazma) geliyor; Ek E'deki 58,9 GB/s ise 614 MB üzerinde, bütün thread'lerle yapılan salt-okuma bir matris-vektör taraması.



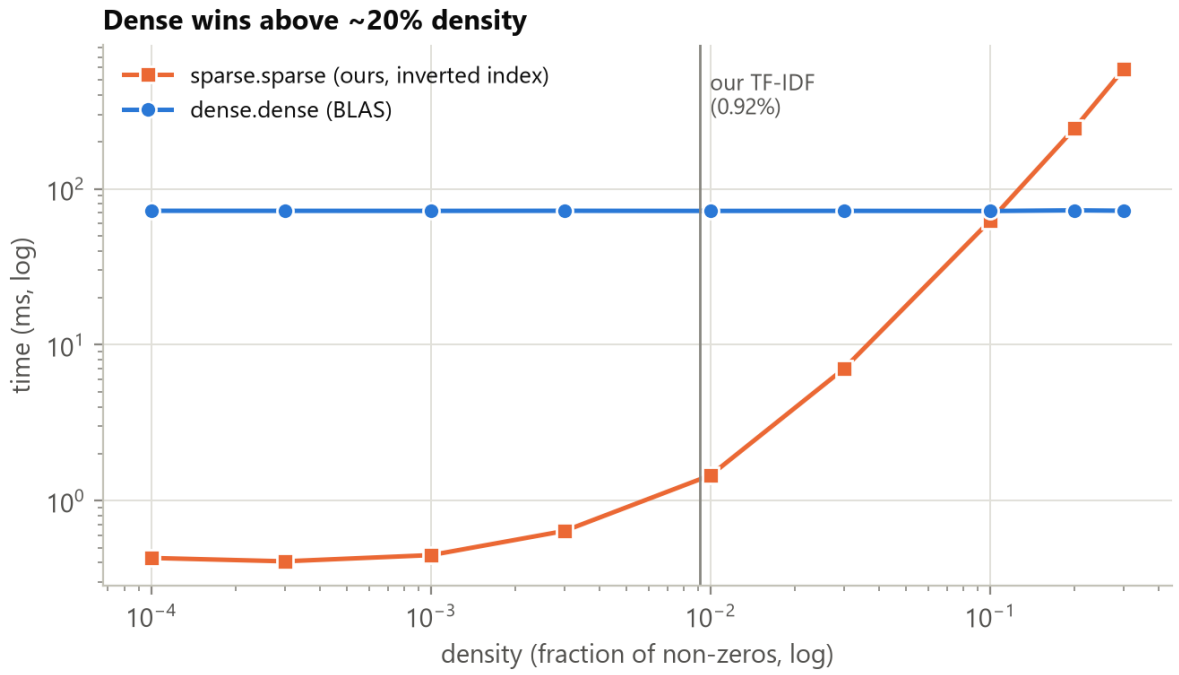
Şekil 9 (bonus): HNSW (FAISS) sorgu başına 0,235 ms'de recall@10 = 0,998'e ulaşıyor; kesin tek thread'li arama 0,537 ms sürüyor. SimHash LSH'ımız 768 boyutta yüksek recall için corpus'un büyük bir kısmını taramak zorunda. 20K dokümanda kesin arama zaten milisaniyenin altında, dolayısıyla ANN ancak çok daha büyük N'de kazandırıyor.

HNSW: indeks 1,67 s'de kuruluyor. Recall 0,904'ten (efSearch = 16, sorgu başına 0,023 ms) 0,9985'e (efSearch = 256, 0,235 ms) çıkıyor.

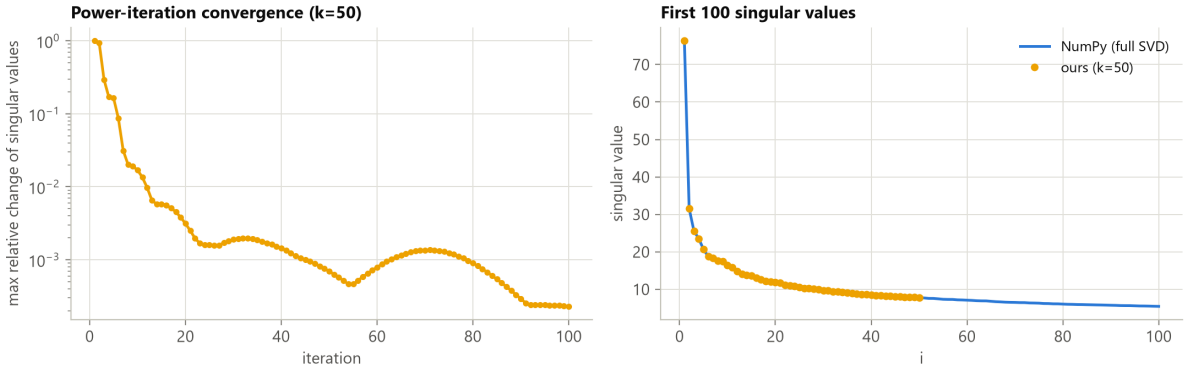
Rastgele hiperdüzlemli LSH'ımız:

- 16 tabloda 8 bitlik kodlarla corpus'un %17,6'sını tarıyor ve 0,62 recall'a ulaşıyor.
- 16 bitlik kodlarda kovaların çoğu neredeyse boş. Sorgu bu durumda tam taramaya geri düşüyor; bu yüzden bu ayar, ortalama aday sayısı çok küçük olduğu halde en yavaşı (sorgu başına 5,1 ms).

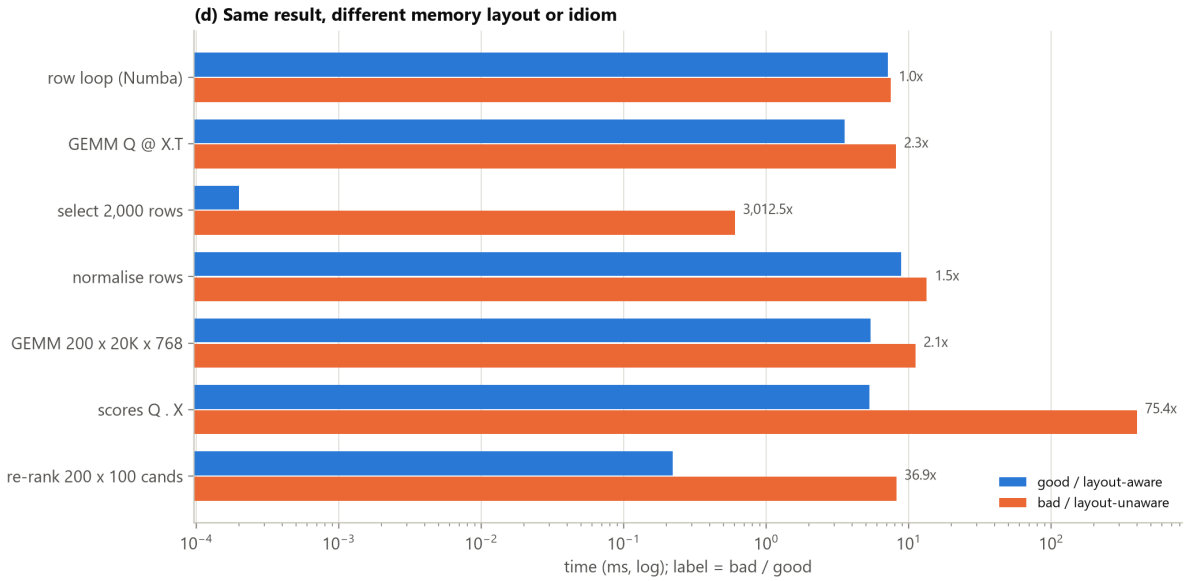
D. Ek şekiller



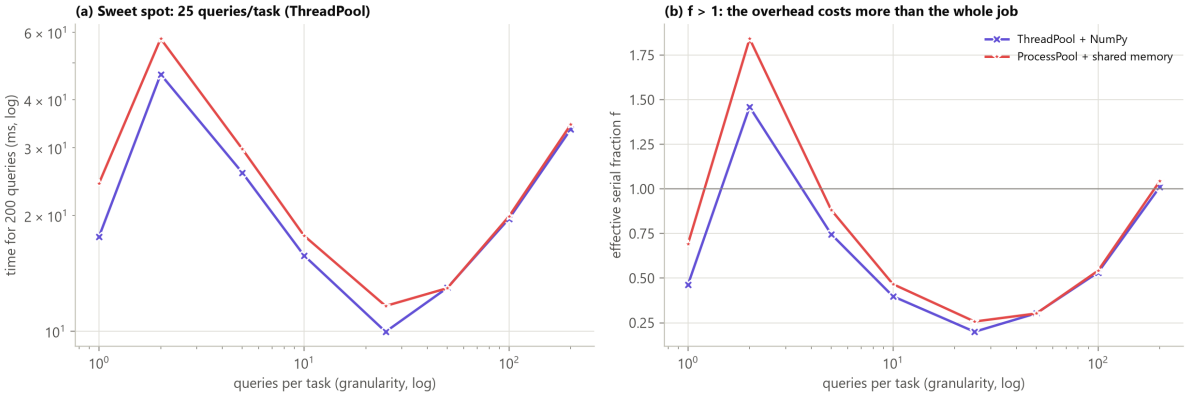
Şekil 2b: Sparse saklama yalnızca çok sparse veride kazandırıyor. Tek thread'li kernel'lerle BLAS dense·dense, %10 ile %20 yoğunluk arasında sparse kernel'imizi geçiyor; TF-IDF matrisimiz ise %0,92 yoğunlukta.



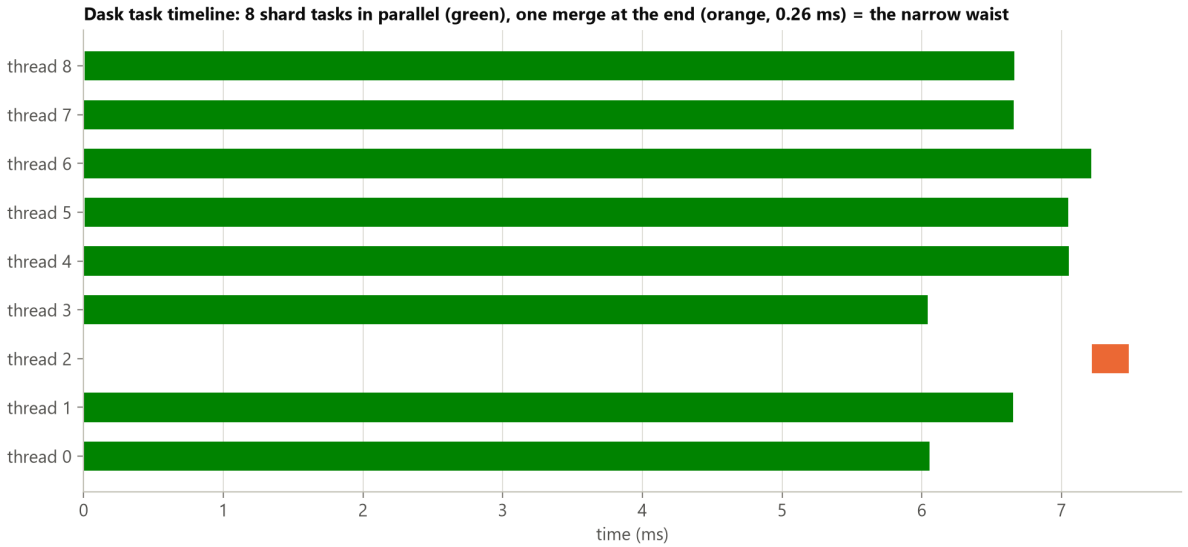
Şekil 3b: Blok power iteration'ımız gerçek tekil değerlere (medyan göreli hata 1,1e-4) ve altuzaya (örtüşme ≥ 0,993) yakınsıyor. Yeniden oluşturma hatası Eckart–Young optimumunun %0,00013'ü içinde.



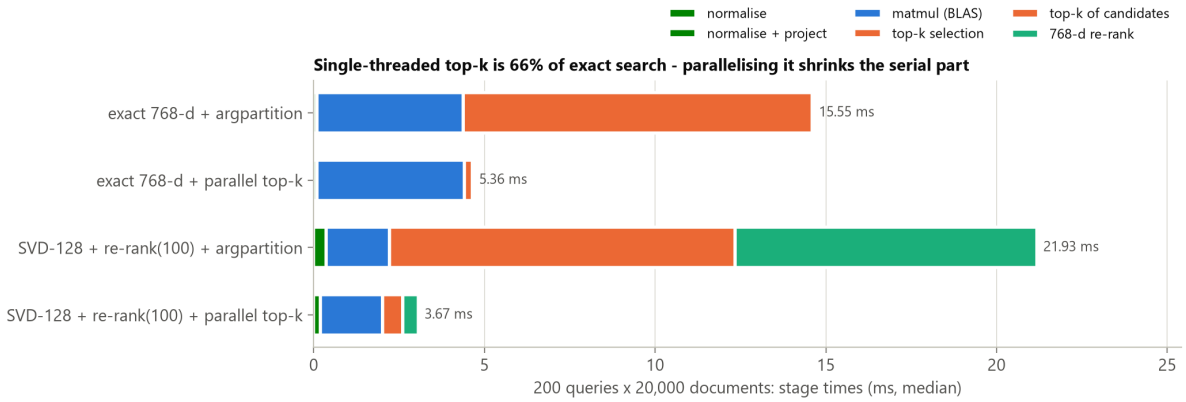
Şekil 4d: Verinin nasıl yerleştirildiği ve nasıl erişildiği aritmetik kadar önemli: fancy-index kopyası slice view'dan 3.012x, BLAS'ı atlayan bir `einsum Q @ X.T`'den 75x, re-rank öncesi aday satırlarını toplamak onları yerinde okumaktan 37x, float64 float32'den 2,1x yavaş. Satır/sütun öncelikli erişim burada yalnızca 1,04x fark ediyor, çünkü 59 MB'lık corpus 96 MB'lık L3 cache'e sığıyor.



Şekil 5c: Granülerlik ana ayar düğmesi. 8 worker ile 200 sorgu en hızlı 25 sorguluk görevler hâlinde çalışıyor (10,0 ms, etkin seri pay f = 0,20). Bütün sorguları içeren tek bir görev diğer worker'ları boşta bırakıyor (33,5 ms). Çok küçük görevler ek yükte boğuluyor: 2 sorguluk görevler (46,6 ms) 1 sorguluk görevlerden (17,6 ms) bile yavaş, çünkü BLAS matris-vektör çarpımından, önce bütün corpus'u paketleyen matris-matris çarpımına geçiyor.



Şekil 5d: Shard'lara bölünmüş tek bir aramanın Dask görev akışı. 8 shard görevi 8 thread'de yan yana çalışıyor (her biri 6,7 ms). Grafın dar boğazı olan tek birleştirme 0,26 ms sürüyor: 7,5 ms'lik toplam (duvar saati) sürenin yalnızca %4'ü; dolayısıyla bu ölçekte düz bir birleştirme yeterli.



Şekil 6a: Kendi motorumuzu profillemek, tek thread'li `argpartition`'ın kesin batch'li aramanın %66'sını aldığını gösteriyor (15,55 ms'nin 10,23'ü): Amdahl yasasındaki seri pay. Paralel bir Numba top-k aramayı 5,36 ms'ye indiriyor; son SVD-128 + re-rank(100) + paralel top-k motoru 3,67 ms sürüyor.

E. 100 milyon dokümanda ölçek duvarları ve hafta 5'in beş sorusu

Duvar	100M dokümanda	Sonuç
Bellek	768-d float32: 307 GB (bu makinenin 33,5 GB'lık RAM'inin 9,2 katı); 128-d: 51 GB (1,5 katı)	Dokümana göre makinelere shard'la.
Tarama süresi	58,9 GB/s'de tek geçiş (ölçüm: 614 MB üzerinde matris-vektör taraması, bütün thread'ler): 5,2 s (768-d), 0,87 s (128-d)	Tam tarama yerine yaklaşık arama (HNSW/IVF, Ek C).
Taşınan veri	9,6 GB'lık 32 shard'ın her biri 163 ms'de taranıyor; bir sorgu her shard'a 3,1 KB gönderip 120 B geri alıyor	Veriyi değil, sorguyu taşı.
Encoding	Bu CPU'da ölçülen saniyede 9,1 özetle: tek makinede 127 gün. %2 seri işle 128 makine yalnızca 36x veriyor: 3,5 gün	Pahalı çevrimdışı adım arama değil, encoding (bir GPU ya da bir API).
Arızalar	Makine başına 1.000 günde bir arızayla: 128 makine x 8 saat için %4,2, 1.000 makine x 24 saat için %63	Yeniden deneme zorunlu; shard görevleri saf (pure) olduğu için yeniden denemek güvenli.

Soru	Motorumuz (20K doküman)	Aynı arama 100M dokümanda
Böl (Split)	Matris çarpımı: BLAS blokları. Top-k ve re-rank: 200 sorgu satırı Numba thread'lerine dağıtılıyor. Corpus bütün kalıyor (59 MB, L3'e sığıyor).	Dokümana göre: corpus'un 32–128 shard'ı, makine başına bir tane.
Ata (Assign)	Statik: BLAS ve OpenMP her thread'e eşit bir blok veriyor (her sorgunun maliyeti aynı).	Shard başına bir görev (Dask / Ray zamanlayıcısı); durumlu actor'lar kendi shard'larını yüklü tutuyor.
Taşı (Move)	Hiçbir şey: paylaşımlı bellek; corpus yerinde okunuyor (pickle yok).	Shard başına 3,1 KB sorgu gidiyor, 120 B (id, skor) dönüyor.
Bekle (Wait)	`prange` sonunda tek bir bariyer; 200 sorguluk batch 3,89 ms sürüyor.	En yavaş shard, ardından shard x k sonucun birleştirilmesi (fan-in büyükse ağaç birleştirme).
Arıza (Fail)	Bir çökme sorguyu yeniden başlatır (milisaniyeler, kaydedilecek bir şey yok).	Shard görevini yeniden dene. Salt-okunur ve saf olduğu için iki kez çalıştırmak güvenli (ölçüldü: %30 başarısız deneme, sonuç yine kesin).

F. İkinci makinede ortak doğrulama ve yeni ölçümler

Ana deneylere ek olarak, aynı 20.000 özet üzerinde Linux ARM64 ortamında bir karşılaştırma daha yaptık. Bu koşuda 4.096 özellikli TF-IDF matrisini TruncatedSVD ile 128 boyuta indirdik ve float64 kullandık. İlk 50 kayıtlı başlığı sorgu olarak aldık; dönüştürücüleri yalnızca özetlerde eğittik. Bu temsil, ana deneylerdeki 768-d MPNet embedding'lerinden farklıdır. Dolayısıyla burada yalnızca aynı makine ve aynı vektörler içindeki hızlanmaları hesaplıyoruz; Windows tablolarıyla doğrudan donanım karşılaştırması yapmıyoruz.

Ortam ve yöntem. Linux aarch64, 4 çekirdek, 23,4 GiB RAM, Python 3.14.4, NumPy 2.5.4, Numba 0.68.0 ve OpenBLAS. 1K, 5K, 10K ve 20K dokümanda sekiz uygulamayı ölçtük: toplam 32 yapılandırma. Her yapılandırma için 3 ısınma çağrısı ve ardından 5 zamanlı tekrar kullandık; toplam 160 zamanlı ölçüm var. Hazırlık ve SVD eğitimi arama süresinin dışında. ThreadPool havuzunu ölçümden önce kurduk; seri baseline için liste dönüşümünü de dışarıda yaptık. Numba matris sürümleri normları arama içinde tekrar hesaplıyor; BLAS ve birleşik kernel hazır birim vektörleri kullanıyor. Bu tablo yalnız derleyici etkisini değil, uygulamaların bütün arama yollarını karşılaştırıyor.

50 sorgu × 20.000 doküman × 128 boyut	Medyan arama (ms)	Aynı ortamda baseline'a göre hızlanma
Saf Python	20.071,953	1,00×
NumPy BLAS, 1 thread	27,430	731,76×
NumPy BLAS, 4 thread	15,343	1.308,21×
Numba seri kosinüs matrisi + seçim	100,737	199,25×
Numba paralel kosinüs matrisi + seçim	46,196	434,50×
Birleşik Numba, 1 thread	35,772	561,11×
Birleşik Numba, 4 thread	18,057	1.111,60×
ThreadPool, 4 worker; worker başına 1 BLAS thread	9,856	2.036,55×

32 karşılaştırmanın tümünde top-10 belge kimlikleri baseline ile birebir eşleşti. En büyük mutlak skor farkı 1,56e-15'in altında kaldı; tolerans 1e-5. Bu koşuda 20K dokümanda en hızlı yöntem ThreadPool oldu. Bu sonuç, Windows/MPNet deneyindeki sıralamanın her donanım ve temsile taşınamayacağını gösteriyor.

Ham beşli süreler, ortalama, standart sapma, veri/kod SHA-256 özetleri ve ortam sürümleri `results/validation_arm64/tfidf_svd_50q/summary.json` içinde; tablolar `timings.csv`, bütün boyutların eğrisi `scaling.png` içinde. Tekrar çalıştırma komutu:

```
OMP_NUM_THREADS=4 OPENBLAS_NUM_THREADS=4 NUMBA_NUM_THREADS=4 \
python scripts/validate_independent.py --output results/validation_arm64/new_run
```

Doğruluk düzeltmeleri. Belge kimliği yanlışken yalnız skor eşitliğiyle başarılı sayılan kontrolleri, sıfır vektörde bölmeyi ve geçersiz k sınırlarını regresyon testleriyle düzelttik. Power iteration SVD'ye altuzay artığına dayalı durdurma ve küçük bir Rayleigh–Ritz SVD sonlandırması ekledik; rank eksikliği ve yakın tekil değerlerde ortogonalliği test ettik. §3.2'deki Windows ölçümleri önceki power-iteration sürümüne aittir; bu düzeltmenin yeni süreleri olarak sunulmuyor. Güncel test komutu `python -m pytest tests -q`.